

Mount Kenya University

Unlocking Infinite Possibilities

Name: CHANDLER MATERE SIMIYU

Admission Number: BSCCS/2024/58571

Unit Title: ADVANCED WEB DESIGN & DEVELOPMENT

Unit Code: BIT3208

Unit Name: Advanced Web Design and Development

Lecturer: MR. NYORO MICHAEL

Year III, Semester II

Project Title: NoteKeeper - Full Stack Web Application

Technologies Used: [Node.js](#) + Express + MySQL

GitHub Link: <https://github.com/Simiyuu/notes-app>

Week 1: Local Environment Setup.....	5
Fig 1: XAMPP Control Panel showing the local development environment installed on the machine.....	5
Fig 2: XAMPP Control Panel showing Apache and MySQL running on their default ports.....	6
Fig 3: Browser showing the XAMPP localhost welcome page.....	6
Fig 4: Hello World Test.....	7
Fig 5: Database Connection Test.....	7
Week 2: Wireframes and GUI Design.....	8
Fig 1: Figma wireframe detailing the planned layout and user interface for the NoteKeeper login page.....	8
Fig 2: Browser showing the Login page with username and password fields used to authenticate registered users.....	9
Fig 3: Mobile View Mockup.....	10
Fig 4: Color Theme and Navigation Structure.....	11
Week 3: JavaScript and PHP Basics.....	12
Fig1: Javascript Form Validation.....	12
Fig 2: Password Strength Checker.....	13
Fig 3: Shows the Express.js route syntax used in server.js to handle HTTP requests.....	14
Fig 4 shows the database connection configuration and initDB function in server.js.....	15
Fig 5: Dynamic User Input Handling.....	15
Week 4 Task Schedule.....	16
Task 1 - Understanding Server-Side Programming.....	16
Fig 1: Simple Processing Page.....	16
Fig 2: Dynamic Welcome Page.....	16
Fig3: Form Submission System.....	17
Task 2 - HTML Forms and PHP Integration.....	18
Fig 1: Registration Form.....	18
Fig 2: Login Form.....	18
Task 3 - Simple Authentication.....	19
Fig 1: Basic Login Page.....	19
Fig 2: Username/passwords validation.....	20
Fig 3: Session-based welcome page.....	21
Task 4 - Backend Folder Organization.....	21
Fig 1: Project Structure.....	21
Task 5 - Introduction to Modern Backend Alternatives.....	22
Fig : PHP Modern Alternative (Node.js).....	22
Week 5 - Advanced Web Design and Development.....	23
Task 1: Database Creation.....	23
Fig 1: notes-app database.....	23
Task 2: Creating Tables.....	23
Fig 1: notes-app tables.....	23

Task 3: CRUD Operations.....	24
Fig 1: Add Records (INSERT).....	24
Fig 2: View Records (SELECT).....	24
Fig 3: Delete Records (DELETE).....	24
Task 4: Connecting PHP to Database.....	25
Fig 1: Database Connection.....	25
Task 5: Fetching Database Records.....	25
Fig 1: Database Records.....	25
Creativity: Note Counter.....	26
Fig 1: shows the dynamic note count badge in the sidebar header displaying the total number of notes.....	26
Week 6: Database Integration and CRUD Operations.....	27
Practical Task 1.....	27
Fig 1: Add a Note (INSERT).....	27
Fig 2: View Notes (SELECT).....	27
Fig 3: Edit a Note (UPDATE).....	28
Fig 4: Delete a Note (DELETE).....	28
Fig 5: Database Export.....	28
Practical Task 2.....	29
Fig 1: Note with Category Added.....	29
Fig 2: Records Stored with Category.....	29
Fig 3: View Records.....	29
Fig 4: Edit a Record.....	30
Fig 5: Delete a Record.....	30
Practical Task 3 (Challenge Task).....	31
Fig 1: User Registration.....	31
Fig 2: User Login.....	31
Fig 3: View Records.....	32
Fig 4: Search Functionality.....	32
Fig 5: Edit Record.....	33
Fig 6: Delete Record.....	33
Fig 7: Responsive Design.....	33
Fig 8: Form Validation.....	34
Week 7: User Authentication and Session Management.....	35
Practical Task 1: Student Portal Login System.....	35
Fig 1: Registration.....	35
Fig 2: Login.....	36
Fig 3: Dashboard.....	37
Fig 4: Logout.....	37
Fig 5: Database Script.....	38
Practical Task 2: Employee Portal.....	

Fig 1: Registration.....	39
Fig 2: Authentication.....	40
Fig 3: Protected Page.....	40
Fig 4: Session Management.....	41
Fig 5: Session Deleted on Logout.....	41
Practical Task 3: Multi-User Authentication System.....	41
Fig 1: Role Selection on Register.....	41
Fig 2: Register Three Users.....	43
Fig 3: Role Badge on Dashboard.....	43
Fig 4: Session Management.....	43
Fig 5: Access Restriction.....	43
Bonus Features.....	44
1. Remember Me: localStorage Keeps Session.....	44
2. Email Verification.....	44
3. Password Reset.....	46
4. 2FA: Google Authenticator.....	49
5. Profile Management.....	51

Week 1: Local Environment Setup

Fig 1: XAMPP Control Panel showing the local development environment installed on the machine

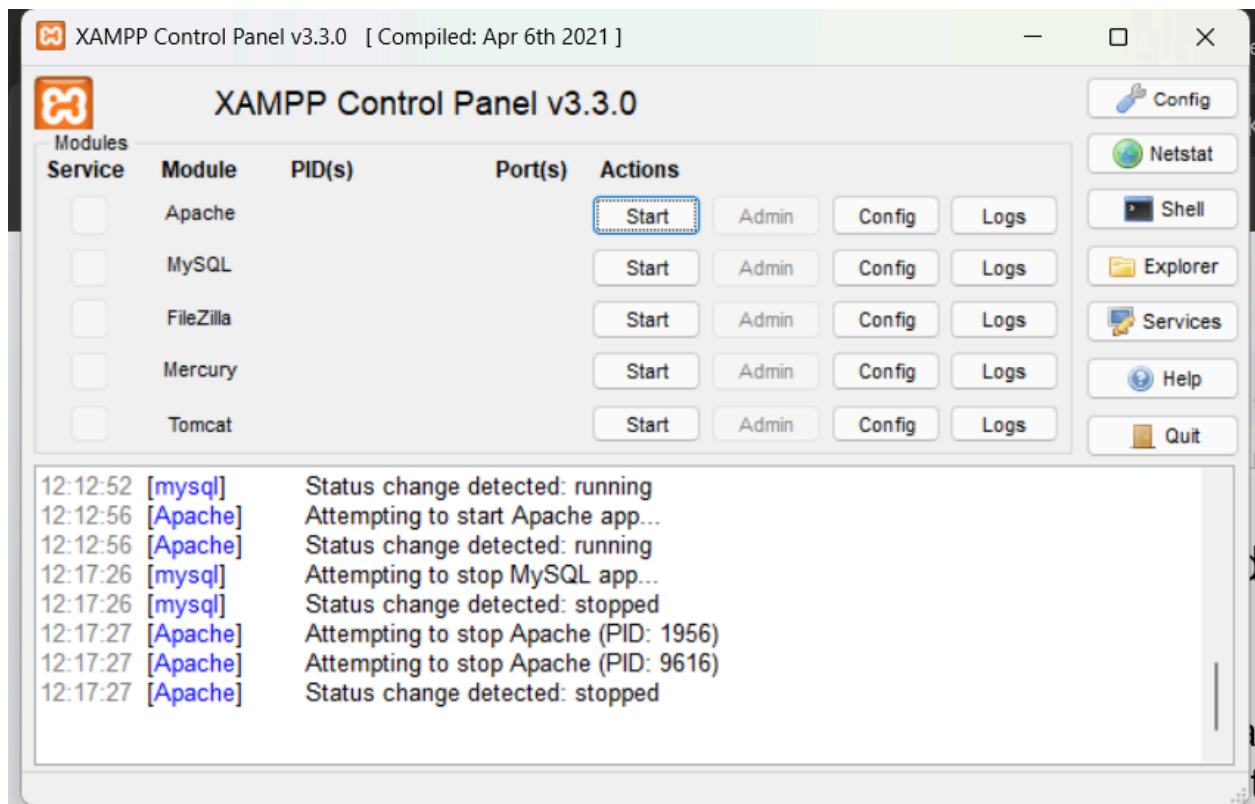


Fig 2: XAMPP Control Panel showing Apache and MySQL running on their default ports

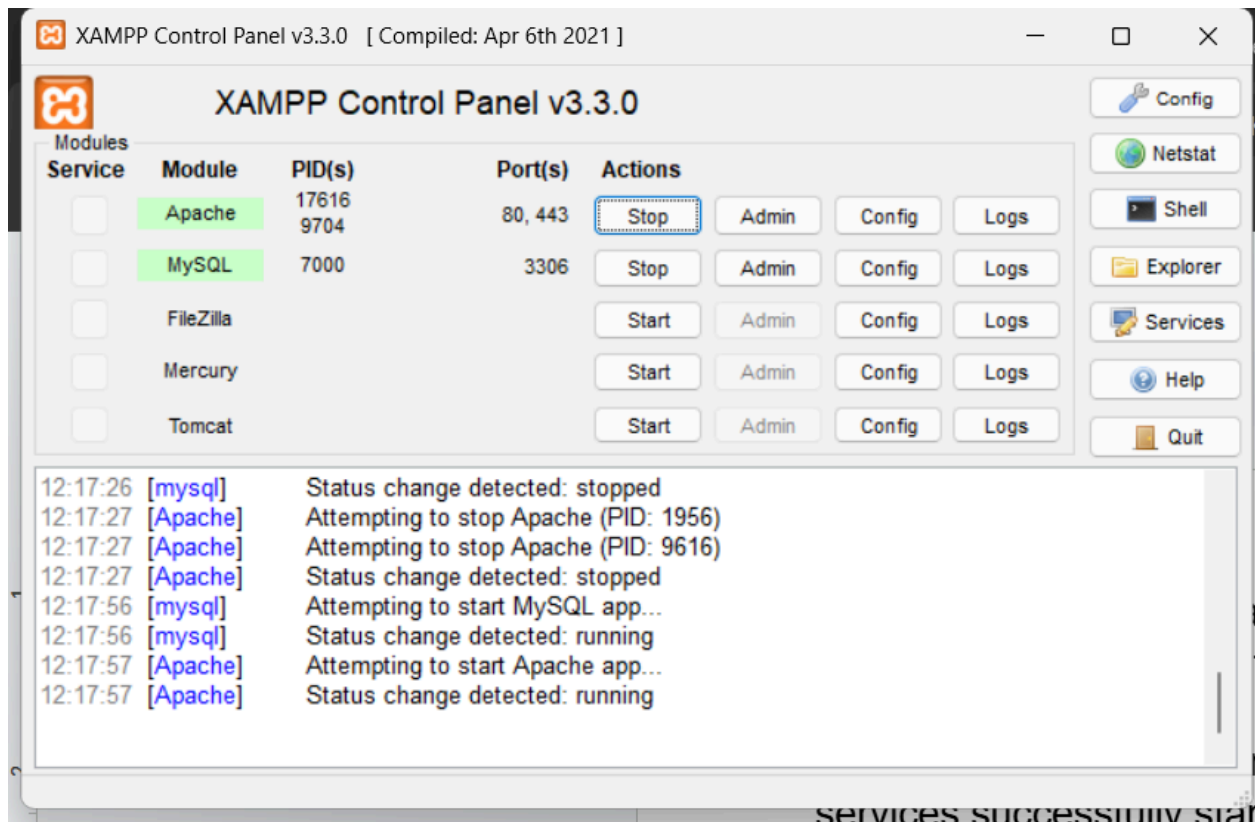


Fig 3: Browser showing the XAMPP localhost welcome page

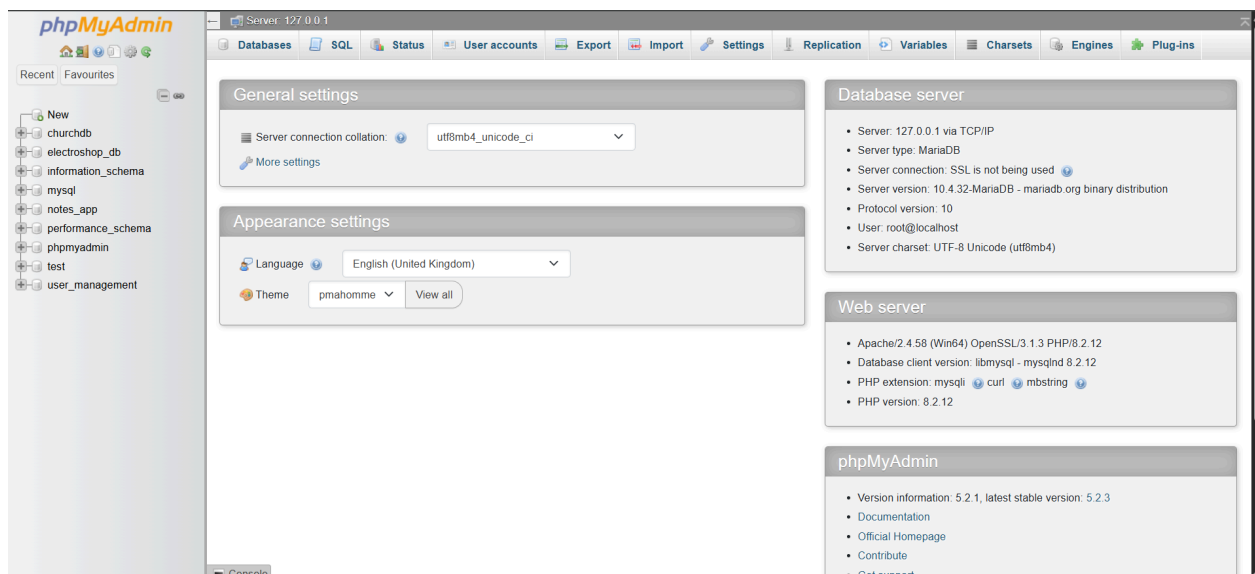


Fig 4: Hello World Test

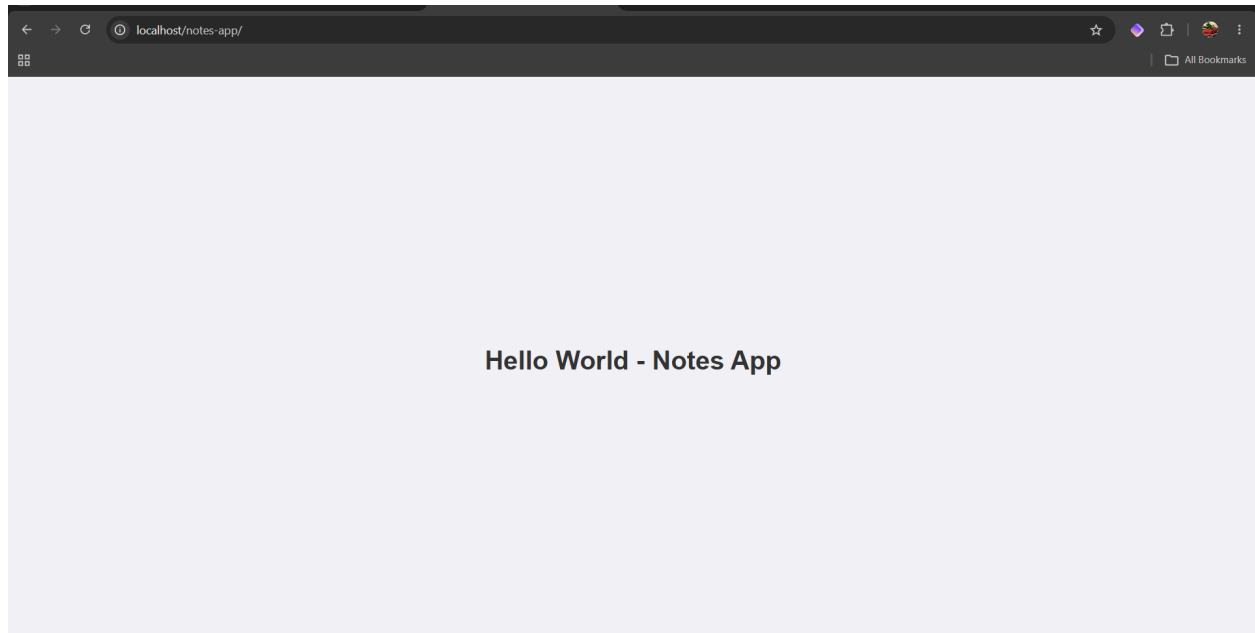
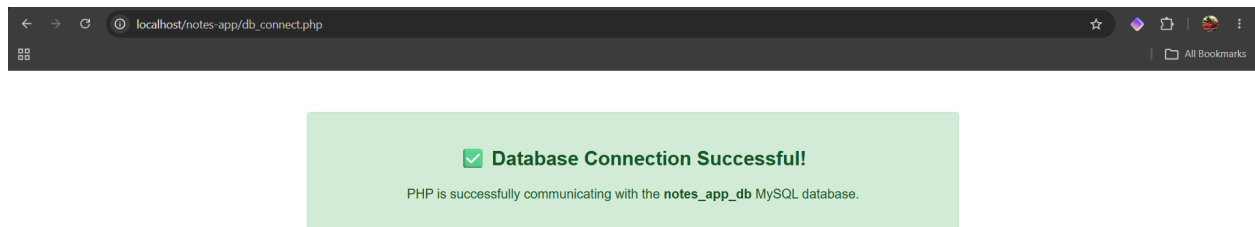


Fig 5: Database Connection Test



Week 2: Wireframes and GUI Design

Fig 1: Figma wireframe detailing the planned layout and user interface for the NoteKeeper login page.

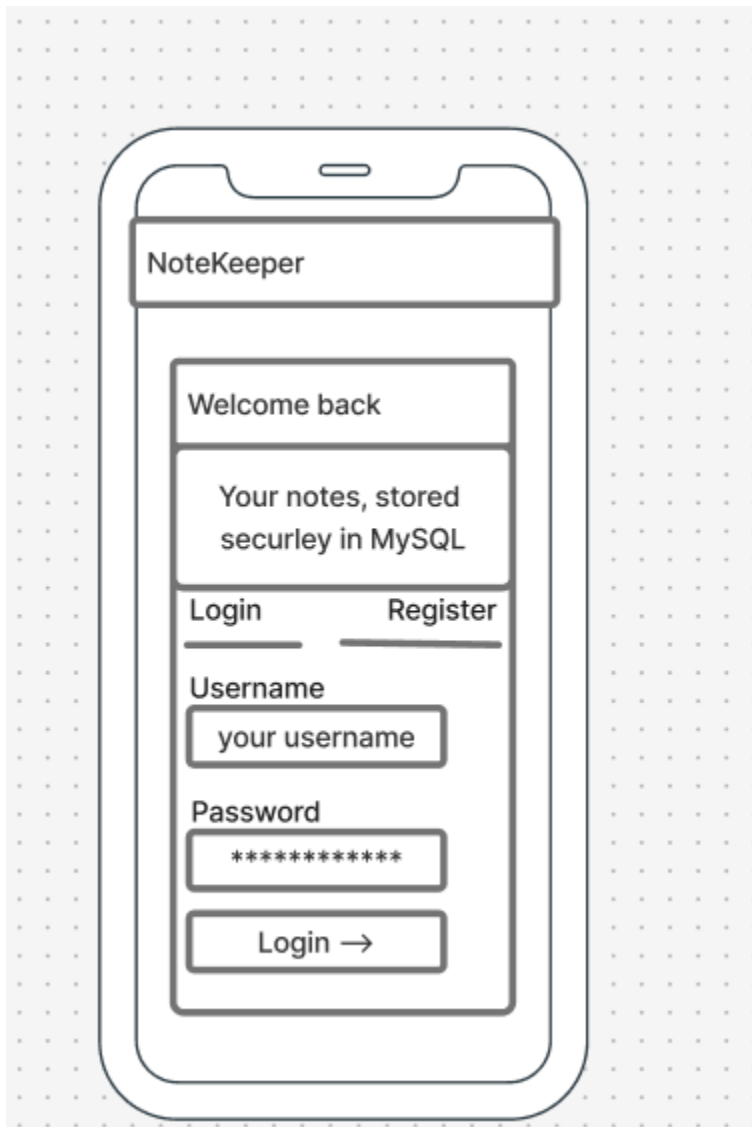
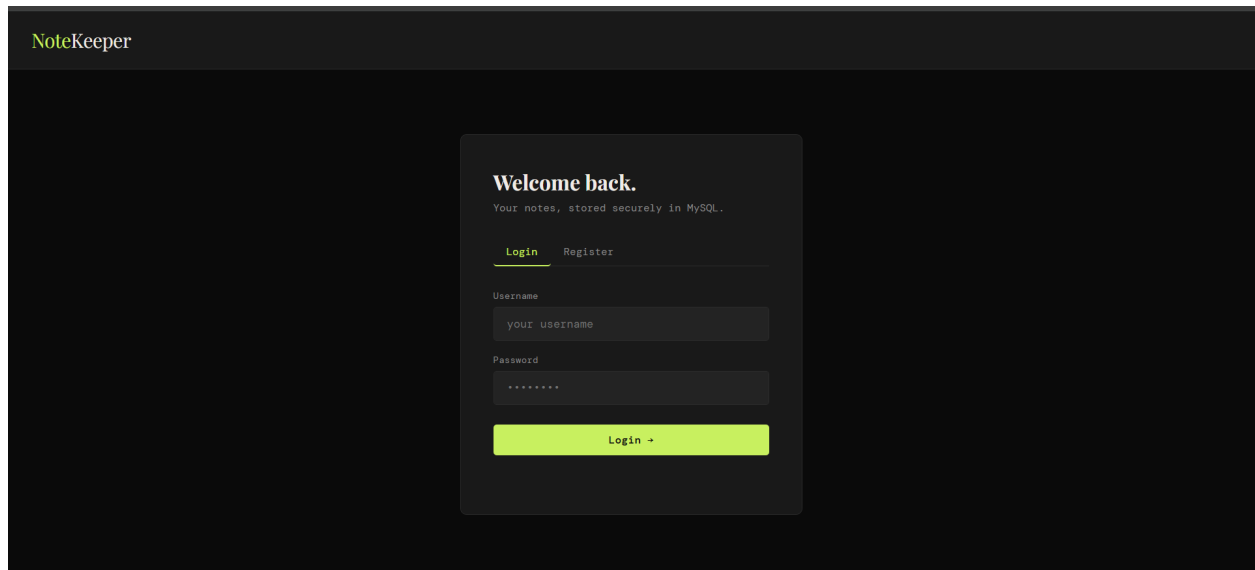


Fig 2: Browser showing the Login page with username and password fields used to authenticate registered users



The image shows a web browser window displaying the login page for an application named "NoteKeeper". The page has a dark theme. At the top left, the "NoteKeeper" logo is visible. The main content area features a central login form with the heading "Welcome back." and a subtext "Your notes, stored securely in MySQL.". Below this, there are two tabs: "Login" (which is active and underlined) and "Register". The "Login" section contains two input fields: "Username" with the placeholder text "your username" and "Password" with masked characters "*****". At the bottom of the form is a bright green button labeled "Login →".

Fig 3: Mobile View Mockup

MY NOTES 2

Read Hacker News
Go to the latest article of The Hacker News
5/28/2026

TryHackMe TODAY
Do labs on TryHackMe to better understanding
5/21/2026

New Note

CREATE NOTE

Title

Note title...

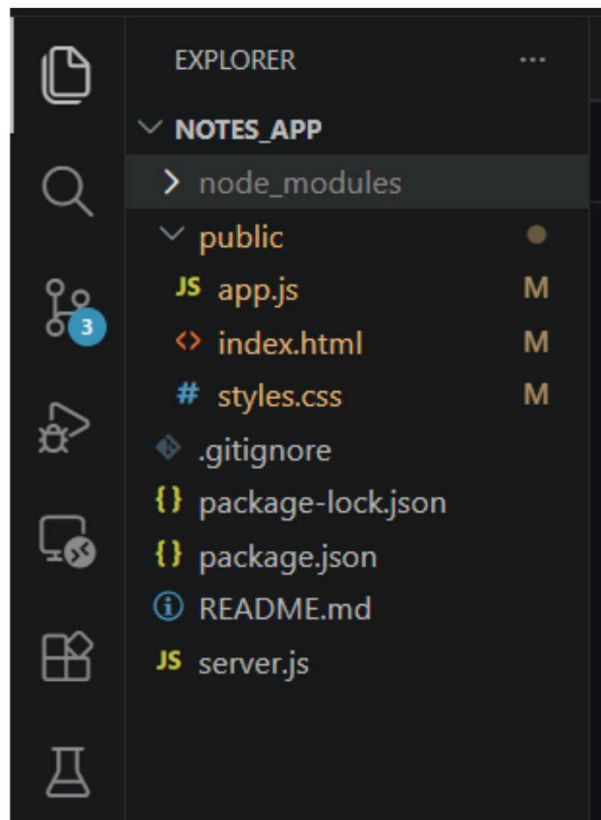
Content

Write your note here...

Save Note

Fig 4: Color Theme and Navigation Structure

	Background
	Surface
	Accent
	Text
	Muted
	Error



Week 3: JavaScript and PHP Basics

Fig1: Javascript Form Validation

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

choose a username

Email

you@example.com

Password

.....

Create Account →

Please fill in all fields

Fig 2: Password Strength Checker

The image shows a dark-themed user interface for a login and registration system. At the top, it says "Welcome back." followed by "Your notes, stored securely in MySQL." Below this are two tabs: "Login" and "Register", with "Register" being the active tab. The form contains three input fields: "Username" with the value "linet", "Email" with the value "linetkamathi@gmail.com", and "Password" which is masked with dots. Below the password field is a strength indicator consisting of a horizontal bar that is approximately 50% filled with orange, and the word "Medium" below it. At the bottom is a large orange button labeled "Create Account →".

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

linet

Email

linetkamathi@gmail.com

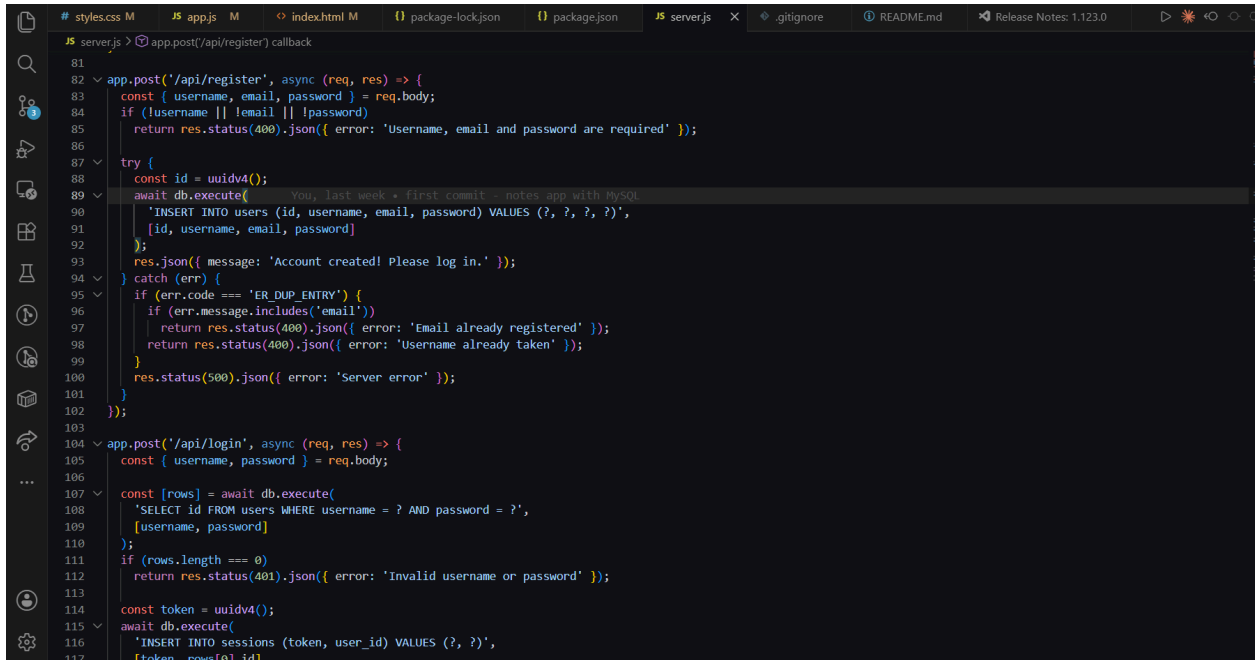
Password

.....

Medium

Create Account →

Fig 3: Shows the Express.js route syntax used in server.js to handle HTTP requests

A screenshot of a code editor with a dark theme. The editor shows a file named 'server.js' with JavaScript code. The code defines two Express.js routes: a POST route for '/api/register' and a POST route for '/api/login'. The '/api/register' route uses 'app.post' and 'async' functions, and it includes database operations for inserting a new user. The '/api/login' route also uses 'app.post' and 'async' functions, and it includes database operations for selecting a user and inserting a session. The code is well-formatted with syntax highlighting and line numbers. The editor's interface includes a sidebar with icons for file explorer, search, and other development tools.

```
81
82 app.post('/api/register', async (req, res) => {
83   const { username, email, password } = req.body;
84   if (!username || !email || !password)
85     return res.status(400).json({ error: 'Username, email and password are required' });
86
87   try {
88     const id = uuidv4();
89     await db.execute(
90       'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
91       [id, username, email, password]
92     );
93     res.json({ message: 'Account created! Please log in.' });
94   } catch (err) {
95     if (err.code === 'ER_DUP_ENTRY') {
96       if (err.message.includes('email'))
97         return res.status(400).json({ error: 'Email already registered' });
98       return res.status(400).json({ error: 'Username already taken' });
99     }
100    res.status(500).json({ error: 'Server error' });
101  }
102 });
103
104 app.post('/api/login', async (req, res) => {
105   const { username, password } = req.body;
106
107   const [rows] = await db.execute(
108     'SELECT id FROM users WHERE username = ? AND password = ?',
109     [username, password]
110   );
111   if (rows.length === 0)
112     return res.status(401).json({ error: 'Invalid username or password' });
113
114   const token = uuidv4();
115   await db.execute(
116     'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
117     [token, rows[0].id]
```

Fig 4 shows the database connection configuration and initDB function in server.js.

```
const DB_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '',
  database: 'notes_app',
  multipleStatements: true
};

let db;

async function initDB() {
  const tempConn = await mysql.createConnection({
    host: DB_CONFIG.host,
    port: DB_CONFIG.port,
    user: DB_CONFIG.user,
    password: DB_CONFIG.password
  });

  await tempConn.execute(`CREATE DATABASE IF NOT EXISTS \`${DB_CONFIG.database}\``);
  await tempConn.end();

  db = await mysql.createPool(DB_CONFIG);
}
```

Fig 5: Dynamic User Input Handling

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds)

SELECT * FROM `notes`

Profiling

[Edit inline]

[Edit]

[Explain SQL]

[Create PHP code]

[Refresh]

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

		id	user_id	title	content	created_at
☐	Edit Copy Delete	14f49b21-31f0-4204-aafd-8f7d2e02a11a	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Machine Learning	How Kenya has integrated Machine Learning	2026-06-03 12:37:30
☐	Edit Copy Delete	9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐	Edit Copy Delete	9f938a0c-f96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14

Week 4 Task Schedule

Task 1 - Understanding Server-Side Programming

Fig 1: Simple Processing Page

```
app.post('/api/register', async (req, res) => {
  const { username, email, password } = req.body;
  if (!username || !email || !password)
    return res.status(400).json({ error: 'Username, email and password are required' });

  try {
    const id = uuidv4();
    await db.execute(
      'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
      [id, username, email, password]
    );
    res.json({ message: 'Account created! Please log in.' });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      if (err.message.includes('email'))
        return res.status(400).json({ error: 'Email already registered' });
      return res.status(400).json({ error: 'Username already taken' });
    }
    res.status(500).json({ error: 'Server error' });
  }
});
```

Fig 2: Dynamic Welcome Page

The screenshot shows the NoteKeeper web application. At the top, the title 'NoteKeeper' is on the left, and 'logged in as chendler' with a 'logout' button is on the right. A sidebar on the left titled 'MY NOTES' shows a count of 3 and lists three items: 'Machine Learning' (dated 07/06/2020), 'Read Hacker News' (dated 07/06/2020), and 'TryHackMe' (dated 07/06/2020). The main content area is titled 'New Note' and contains a 'CREATE NOTE' form. The form has two input fields: 'Title' with placeholder text 'Note title...' and 'Content' with placeholder text 'Write your note here...'. A green 'Save Note' button is at the bottom of the form.

Fig3: Form Submission System

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

linet

Email

linetkamathi@gmail.com

Password

.....

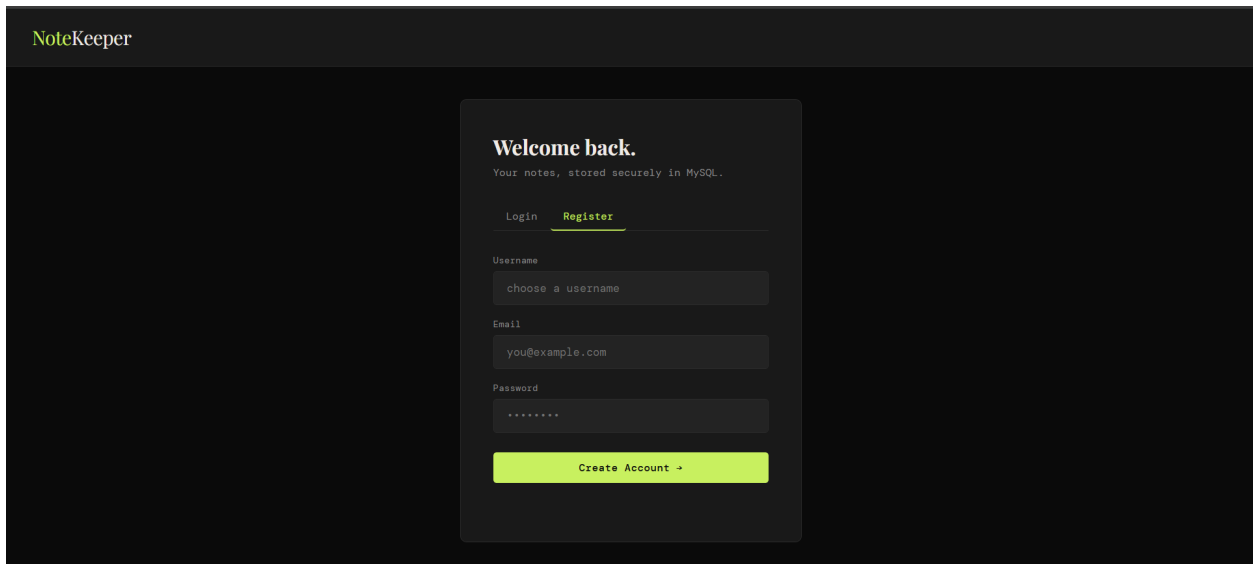
Medium

Create Account →

✓ Account created! Please log in.

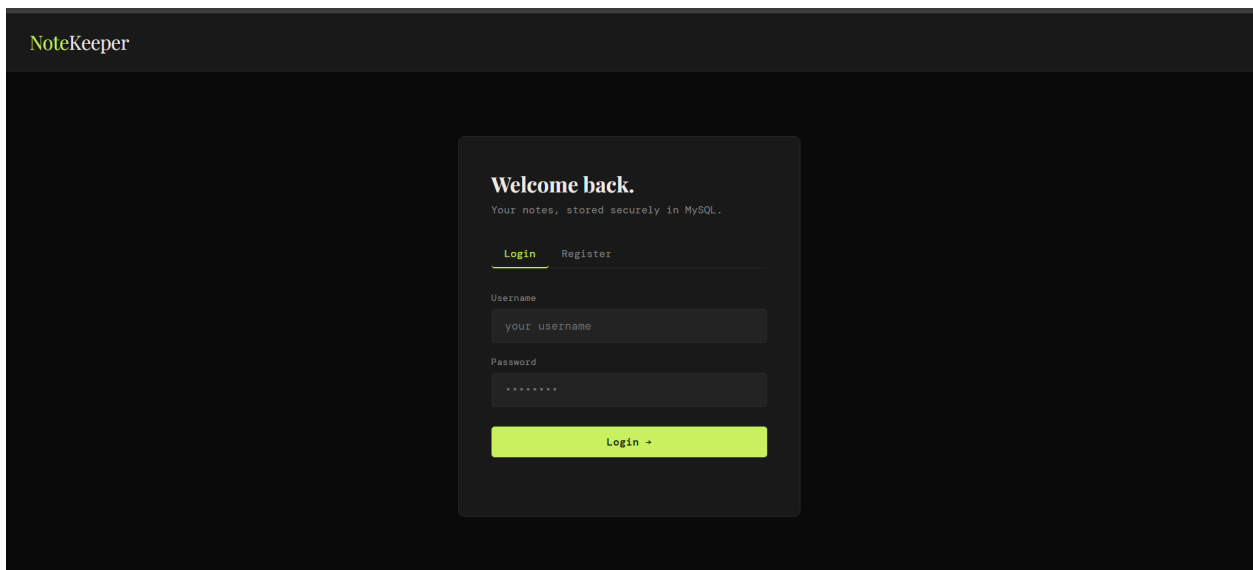
Task 2 - HTML Forms and PHP Integration

Fig 1: Registration Form



The screenshot shows the registration form of the NoteKeeper application. The header is dark gray with the 'NoteKeeper' logo in green and white. The main content area is black, featuring a central white card. The card has a heading 'Welcome back.' and a subtext 'Your notes, stored securely in MySQL.'. Below this, there are two tabs: 'Login' and 'Register', with 'Register' being the active tab. The form fields include 'Username' with a placeholder 'choose a username', 'Email' with a placeholder 'you@example.com', and 'Password' with a placeholder '*****'. A green button labeled 'Create Account →' is positioned at the bottom of the form.

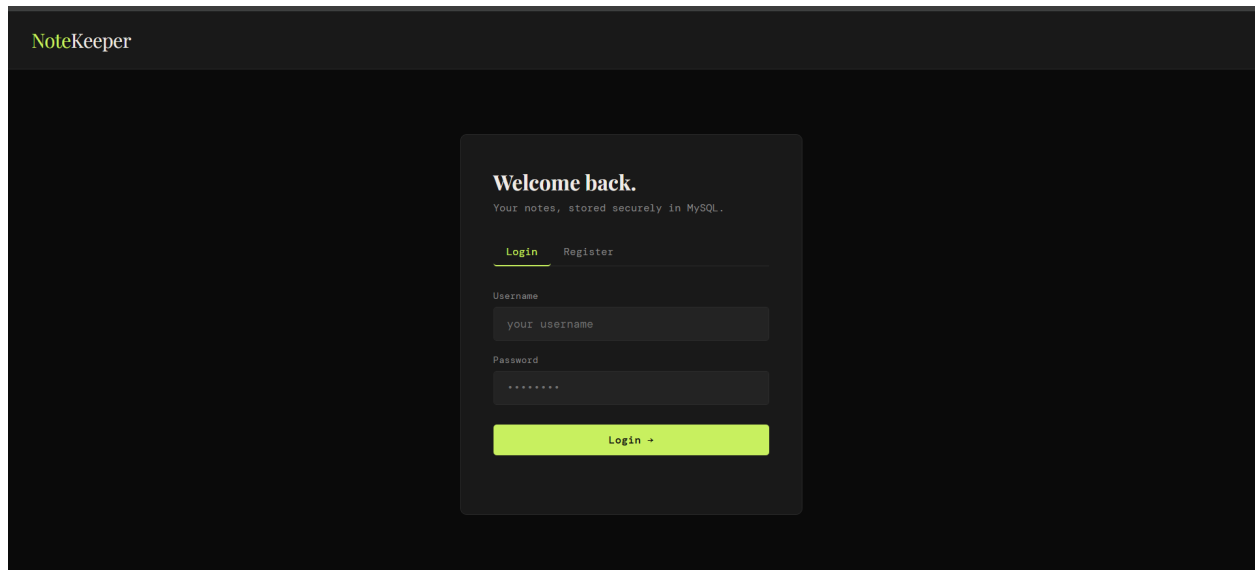
Fig 2: Login Form



The screenshot shows the login form of the NoteKeeper application. The header is dark gray with the 'NoteKeeper' logo in green and white. The main content area is black, featuring a central white card. The card has a heading 'Welcome back.' and a subtext 'Your notes, stored securely in MySQL.'. Below this, there are two tabs: 'Login' and 'Register', with 'Login' being the active tab. The form fields include 'Username' with a placeholder 'your username' and 'Password' with a placeholder '*****'. A green button labeled 'Login →' is positioned at the bottom of the form.

Task 3 - Simple Authentication

Fig 1: Basic Login Page



The image shows a web application interface for 'NoteKeeper'. The header is dark grey with the 'NoteKeeper' logo in the top left corner. The main content area is black and features a central white login card. The card has a heading 'Welcome back.' followed by the text 'Your notes, stored securely in MySQL.'. Below this, there are two links: 'Login' (underlined) and 'Register'. The 'Login' section includes a 'Username' label and a text input field containing 'your username', and a 'Password' label and a password input field with masked characters. At the bottom of the card is a large blue button labeled 'Login →'.

NoteKeeper

Welcome back.

Your notes, stored securely in MySQL.

[Login](#) [Register](#)

Username

Password

Login →

Fig 2: Username/passwords validation

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

linet

Email

linetkamathi@gmail.com

Password

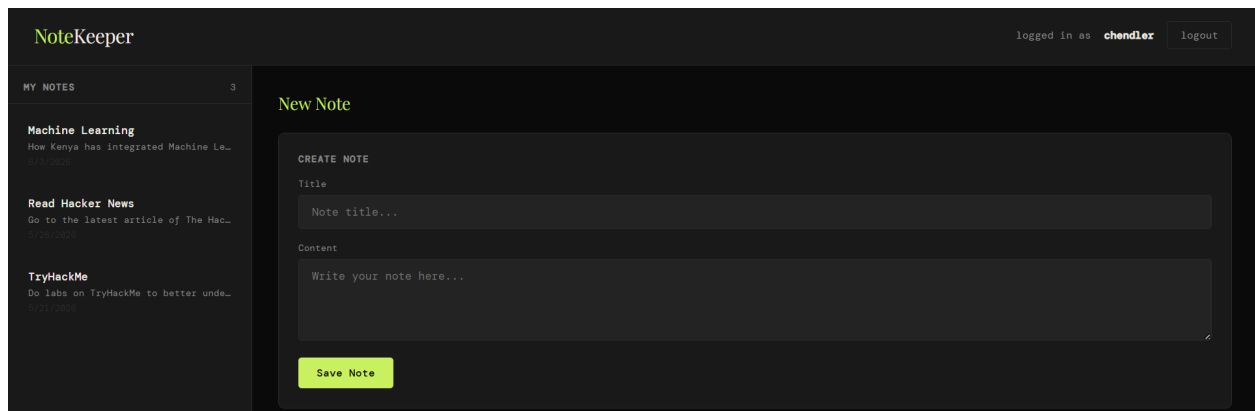
.....

Medium

Create Account →

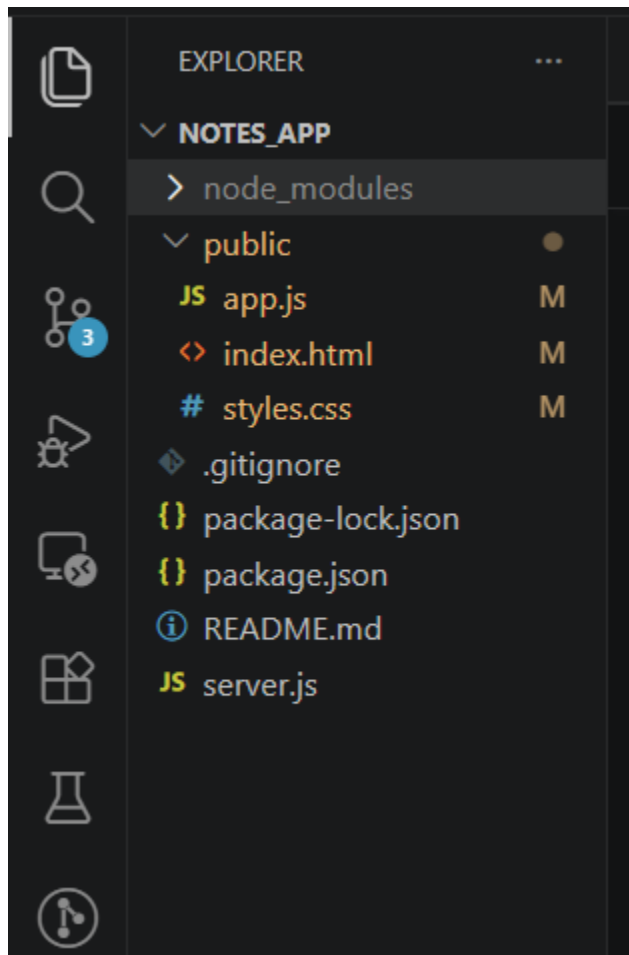
✓ Account created! Please log in.

Fig 3: Session-based welcome page



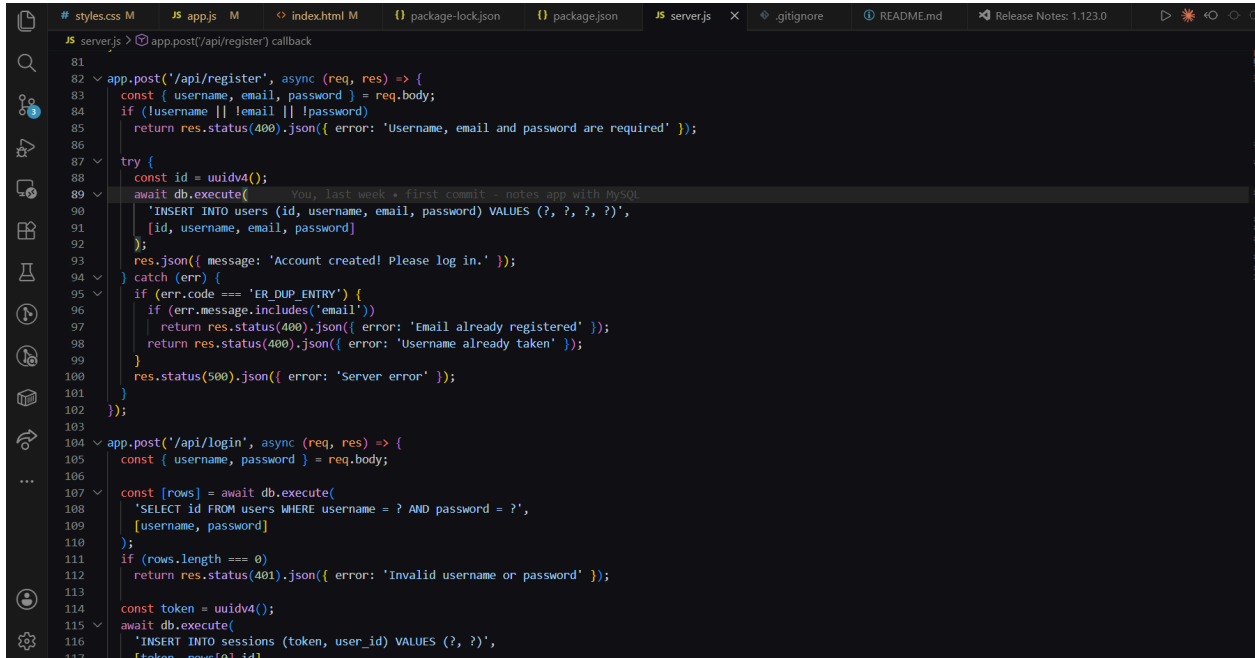
Task 4 - Backend Folder Organization

Fig 1: Project Structure



Task 5 - Introduction to Modern Backend Alternatives

Fig : PHP Modern Alternative ([Node.js](#))



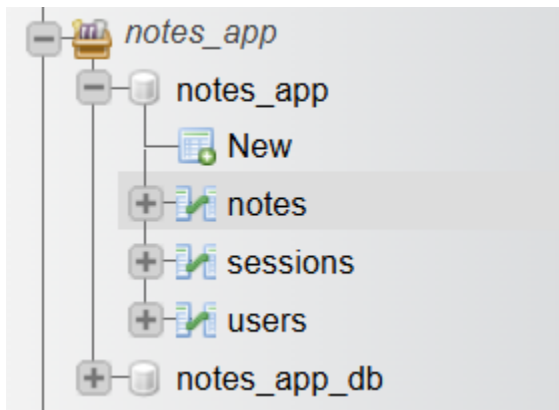
The image shows a code editor window with a dark theme. The top bar displays several open files: # styles.css M, JS app.js M, index.html M, package-lock.json, package.json, JS server.js, .gitignore, README.md, and Release Notes: 1.123.0. The main editor area shows the content of 'server.js'. The code is written in JavaScript and uses Express.js for handling HTTP requests. It includes a registration endpoint (app.post('/api/register')) and a login endpoint (app.post('/api/login')). The registration endpoint checks if the username, email, and password are provided. If not, it returns a 400 status with an error message. If provided, it generates a UUID for the user ID and inserts the user into a database. It then checks for duplicate entries (email or username) and returns appropriate error messages. If successful, it returns a 200 status with a success message. The login endpoint checks if the username and password are provided. If not, it returns a 400 status with an error message. If provided, it queries the database for the user. If found, it generates a token and inserts it into a sessions table. If not found, it returns a 401 status with an error message. The code uses async/await for database operations.

```
81
82 ✓ app.post('/api/register', async (req, res) => {
83   const { username, email, password } = req.body;
84   if (!username || !email || !password)
85     return res.status(400).json({ error: 'Username, email and password are required' });
86
87   try {
88     const id = uuidv4();
89     await db.execute(
90       'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
91       [id, username, email, password]
92     );
93     res.json({ message: 'Account created! Please log in.' });
94   } catch (err) {
95     if (err.code === 'ER_DUP_ENTRY') {
96       if (err.message.includes('email'))
97         return res.status(400).json({ error: 'Email already registered' });
98       return res.status(400).json({ error: 'Username already taken' });
99     }
100    res.status(500).json({ error: 'Server error' });
101  }
102 });
103
104 ✓ app.post('/api/login', async (req, res) => {
105   const { username, password } = req.body;
106
107   const [rows] = await db.execute(
108     'SELECT id FROM users WHERE username = ? AND password = ?',
109     [username, password]
110   );
111   if (rows.length === 0)
112     return res.status(401).json({ error: 'Invalid username or password' });
113
114   const token = uuidv4();
115   await db.execute(
116     'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
117     [token, rows[0].id]
```

Week 5 - Advanced Web Design and Development

Task 1: Database Creation

Fig 1: notes-app database



Task 2: Creating Tables

Fig 1: notes-app tables

Table	Action	Rows	Type	Collation	Size	Overhead
☐ notes	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ sessions	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	-
3 tables	Sum	8	InnoDB	utf8mb4_general_ci	112.0 KiB	0 B

Task 3: CRUD Operations

Fig 1: Add Records (INSERT)

+ -				user_id	title	content	created_at	
☐				14f49b21-31f0-4204-aafd-8f7d2e02a11a	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Machine Learning	How Kenya has integrated Machine Learning	2026-06-03 12:37:30
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14
☐ Check all With selected: Edit Copy Delete Export								

Fig 2: View Records (SELECT)

NoteKeeper

MY NOTES3

Machine Learning

How Kenya has integrated Machine Le...

Read Hacker News

Go to the latest article of The Hac...

5/28/2026

TryHackMe

Do labs on TryHackMe to better unde...

5/21/2026

New Note

CREATE NOTE

Title

Note title...

Content

Write your note here...

Save Note

Fig 3: Delete Records (DELETE)

				id	user_id	title	content	created_at
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14

Fig 4: Edit Records (UPDATE)

				id	user_id	title	content	created_at
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe TODAY	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14
☐ Check all With selected: Edit Copy Delete Export								

Task 4: Connecting PHP to Database

Fig 1: Database Connection

```
const DB_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '',
  database: 'notes_app',
  multipleStatements: true
};

let db;

async function initDB() {
  const tempConn = await mysql.createConnection({
    host: DB_CONFIG.host,
    port: DB_CONFIG.port,
    user: DB_CONFIG.user,
    password: DB_CONFIG.password
  });

  await tempConn.execute(`CREATE DATABASE IF NOT EXISTS \`${DB_CONFIG.database}\``);
  await tempConn.end();

  db = await mysql.createPool(DB_CONFIG);
}
```

Task 5: Fetching Database Records

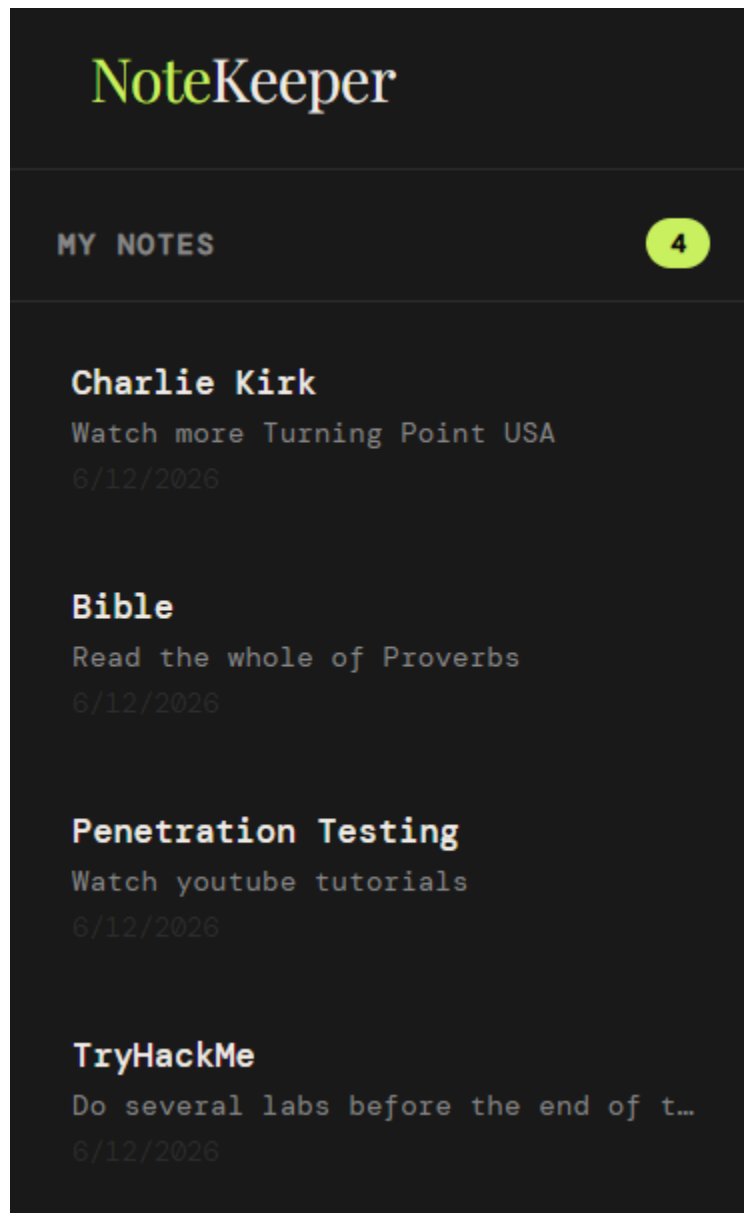
Fig 1: Database Records

```
app.put('/api/notes/:id', requireAuth, async (req, res) => {
  const { title, content } = req.body;
  if (!title || !content)
    return res.status(400).json({ error: 'Title and content required' });

  await db.execute(
    'UPDATE notes SET title = ?, content = ? WHERE id = ? AND user_id = ?',
    [title, content, req.params.id, req.userId]
  );
  res.json({ message: 'Note updated!' });
});
```

Creativity: Note Counter

Fig 1: shows the dynamic note count badge in the sidebar header displaying the total number of notes.



Week 6: Database Integration and CRUD Operations

Practical Task 1

Fig 1: Add a Note (INSERT)

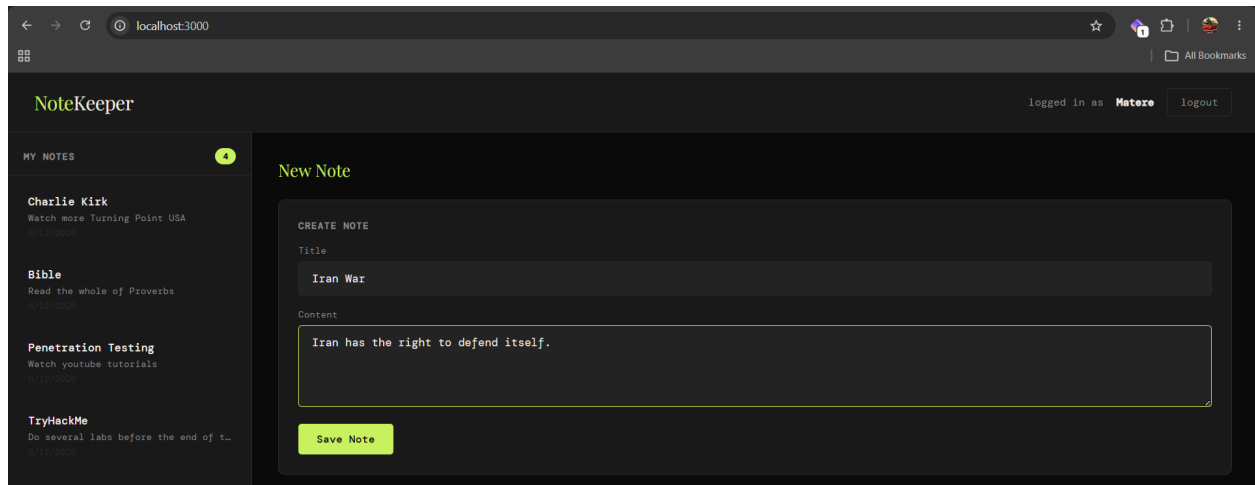


Fig 2: View Notes (SELECT)

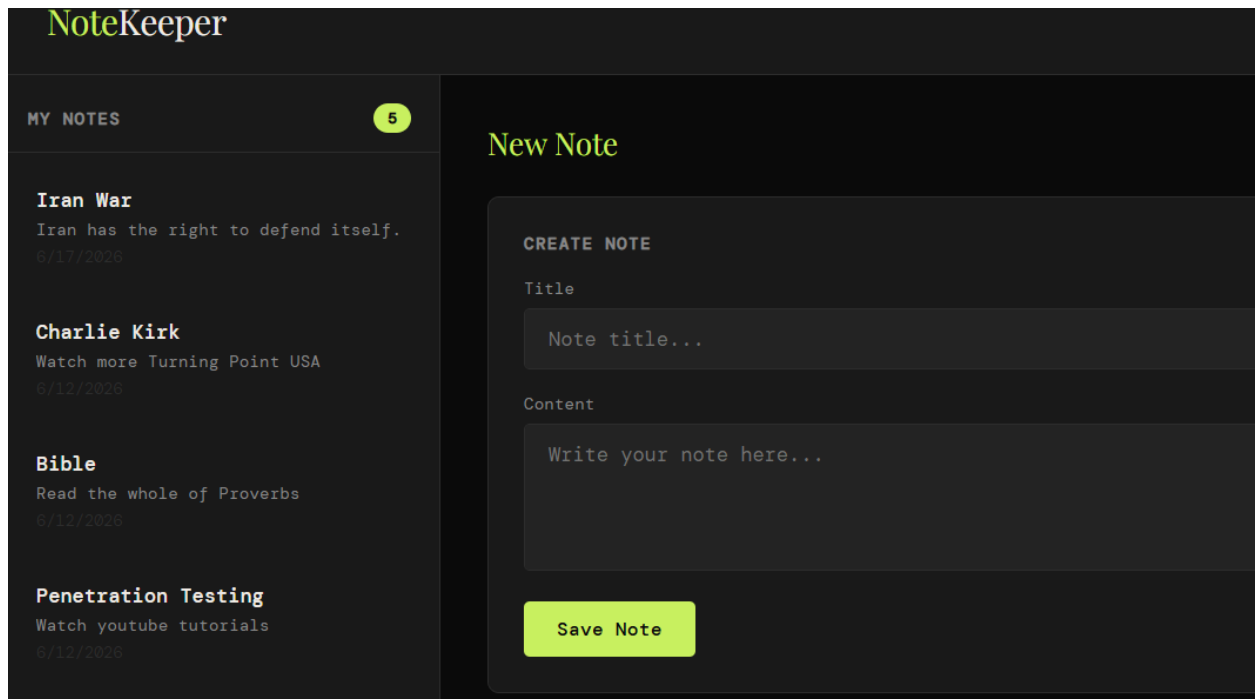


Fig 3: Edit a Note (UPDATE)

New Note

CREATE NOTE

Title

Pen Tester

Content

Watch youtube tutorials

Update Note

Fig 4: Delete a Note (DELETE)

		id	user_id	title	content	created_at
☐	 Edit  Copy  Delete	44de630b-b69d-4329-9c6f-ad2a23efb42e	954c422b-6eb7-486f-92ed-99341205d3f2	Charlie Kirk	Watch more Turning Point USA	2026-06-12 04:53:18
☐	 Edit  Copy  Delete	6daa1bf7-84f3-41fa-af38-648203afca9f	954c422b-6eb7-486f-92ed-99341205d3f2	TryHackMe	Do several labs before the end of the week.	2026-06-12 04:50:01
☐	 Edit  Copy  Delete	7fe950ea-1a33-4bae-9cce-cf6e33656dca	954c422b-6eb7-486f-92ed-99341205d3f2	Iran War	Iran has the right to defend itself.	2026-06-17 07:56:39
☐	 Edit  Copy  Delete	bd2baec9-358b-467f-88dd-bd13f521a874	954c422b-6eb7-486f-92ed-99341205d3f2	Bible	Read the whole of Proverbs	2026-06-12 04:52:54

Fig 5: Database Export

```
> week1
> week2
> week3
> week4
v week5
  JS server.js
  .gitignore
  Notes Application Docum...
  notes_app.sql
  {} package-lock.json
71
72
73
74
75
76
77
78
79
80
81
82
<div class= sidebar
  <div class="sideba
    <h3>My Notes</h3>
    <span id="noteCo
  </div>
  <div class="notes-
    <div class="empt
  </div>
</div>
<div class="main-par
  <div class="panel-
```

Practical Task 2

Fig 1: Note with Category Added

New Note

CREATE NOTE

Title

Cipher Text

Category

Study

Content

Why is it called Caesar Cipher.

Save Note

Fig 2: Records Stored with Category

		id	user_id	title	content	category	created_at
	Edit Copy Delete	065373d4-ef12-453c-9a9a-43778fa81694	954c422b-6eb7-486f-92ed-99341205d3f2	Bible Study	Join the Bible study today at 8pm.	Personal	2026-06-17 08:29:58
	Edit Copy Delete	38e99915-e346-424c-857e-4fac777a9a82	954c422b-6eb7-486f-92ed-99341205d3f2	Drink More Water	Water helps with everyday tasks	General	2026-06-17 08:28:51
	Edit Copy Delete	4e52ad21-953c-494e-8829-fae1206837c6	954c422b-6eb7-486f-92ed-99341205d3f2	Tech Companies	Research about several tech companies in Nairobi.	Work	2026-06-17 08:29:36
	Edit Copy Delete	820f7cd8-5ae5-4865-83ec-692f802c00f3	954c422b-6eb7-486f-92ed-99341205d3f2	Penetration Testing	Watch tutorials on how to Pen Test.	Study	2026-06-17 08:30:30
Check all With selected: Edit Copy Delete Export							

Fig 3: View Records

MY NOTES

4

Penetration Testing

Watch tutorials on how to Pen Test.

07/17/2026

Bible Study

Join the Bible study today at 8pm.

07/17/2026

Tech Companies

Research about several tech compani...

07/17/2026

Drink More Water

Water helps with everyday tasks.

07/17/2026

New Note

CREATE NOTE

Title

Note title...

Category

Study

Content

Write your note here...

Save Note

Fig 4: Edit a Record

New Note

CREATE NOTE

Title

Tech Companies & Software

Category

Study

Content

Research about several tech companies in Nairobi.

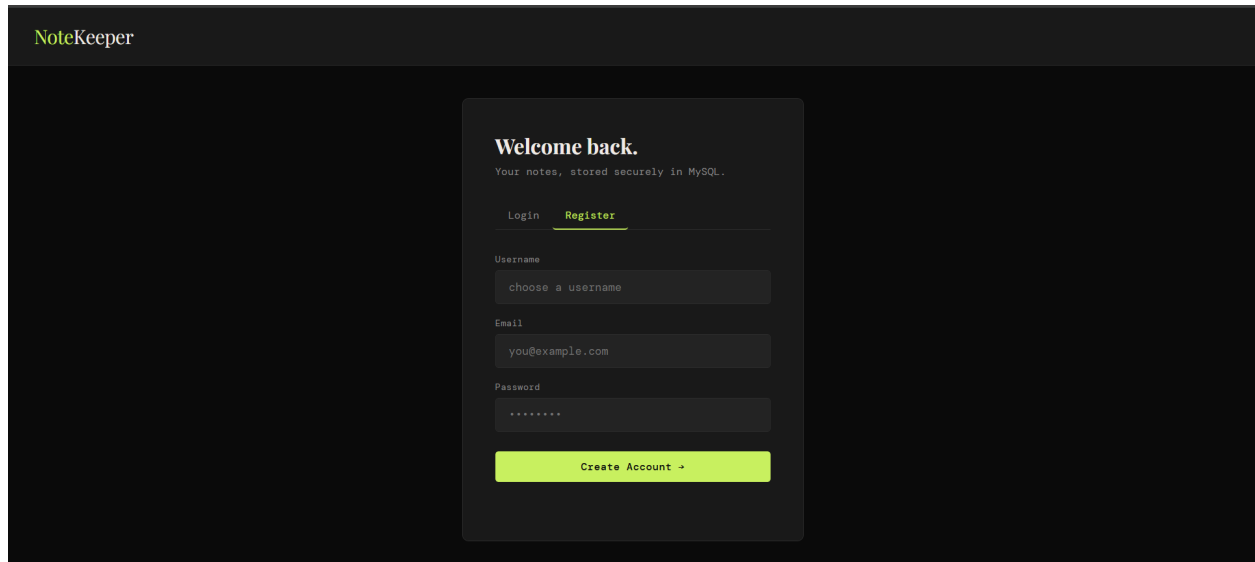
Update Note

Fig 5: Delete a Record

		id	user_id	title	content	category	created_at
☐	 Edit  Copy  Delete	065373d4-ef12-453c-9a9a-43778fa81694	954c422b-6eb7-486f-92ed-99341205d3f2	Bible Study	Join the Bible study today at 8pm.	Personal	2026-06-17 08 29 58
☐	 Edit  Copy  Delete	4e52ad21-953c-494e-8829-fae1206837c6	954c422b-6eb7-486f-92ed-99341205d3f2	Tech Companies & Software	Research about several tech companies in Nairobi.	Work	2026-06-17 08 29 36
☐	 Edit  Copy  Delete	820f7cd8-5ae5-4865-83ec-692f802c00f3	954c422b-6eb7-486f-92ed-99341205d3f2	Penetration Testing	Watch tutorials on how to Pen Test.	Study	2026-06-17 08 30 30
		☐ Check all With selected:  Edit  Copy  Delete  Export					

Practical Task 3 (Challenge Task)

Fig 1: User Registration



The image shows a user registration form for a web application named "NoteKeeper". The form is centered on a dark background. At the top left, the "NoteKeeper" logo is displayed. The form itself has a title "Welcome back." and a subtitle "Your notes, stored securely in MySQL.". Below this, there are two tabs: "Login" and "Register", with "Register" being the active tab. The form contains three input fields: "Username" with a placeholder "choose a username", "Email" with a placeholder "you@example.com", and "Password" with a placeholder "*****". A green "Create Account ->" button is located at the bottom of the form.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

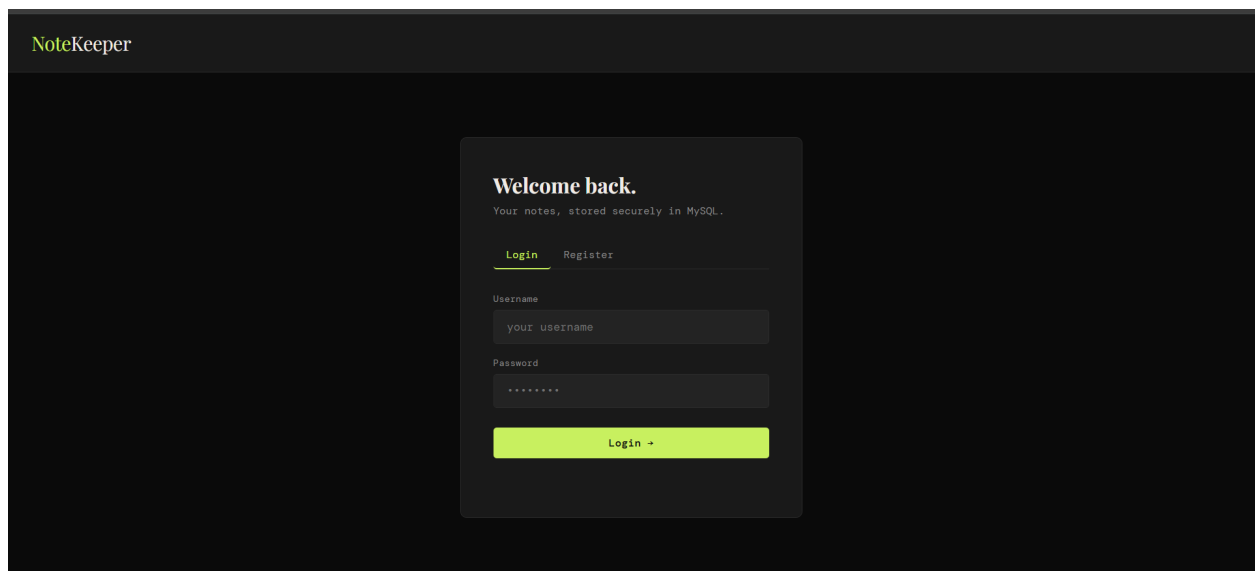
Username
choose a username

Email
you@example.com

Password

Create Account ->

Fig 2: User Login



The image shows a user login form for the same "NoteKeeper" web application. The form is centered on a dark background. At the top left, the "NoteKeeper" logo is displayed. The form has a title "Welcome back." and a subtitle "Your notes, stored securely in MySQL.". Below this, there are two tabs: "Login" and "Register", with "Login" being the active tab. The form contains two input fields: "Username" with a placeholder "your username" and "Password" with a placeholder "*****". A green "Login ->" button is located at the bottom of the form.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

Username
your username

Password

Login ->

Fig 3: View Records

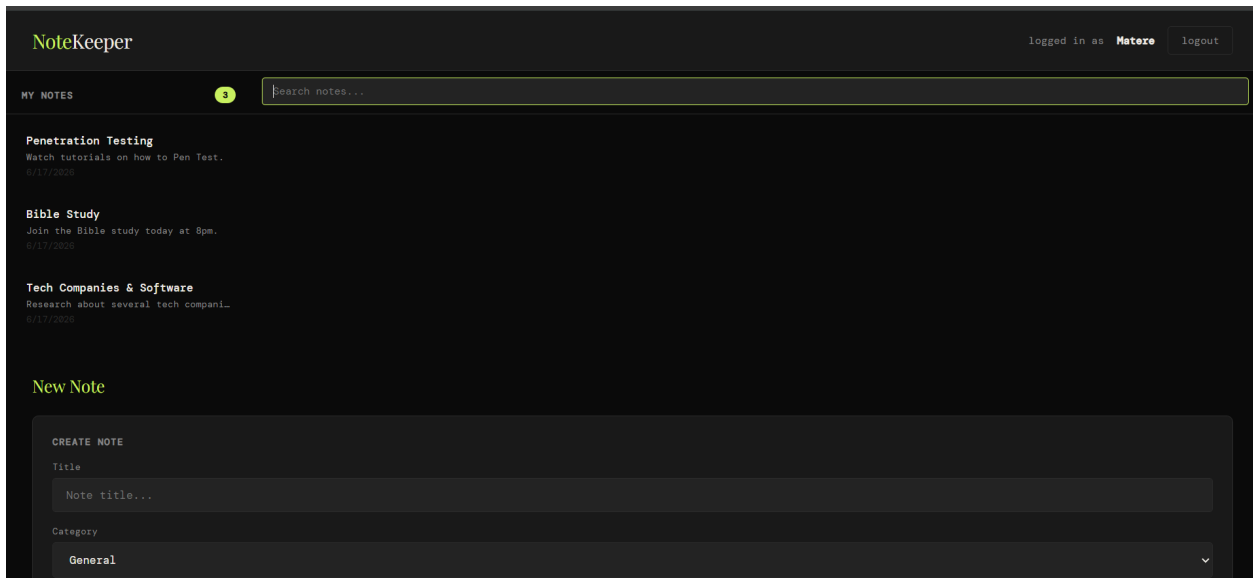


Fig 4: Search Functionality

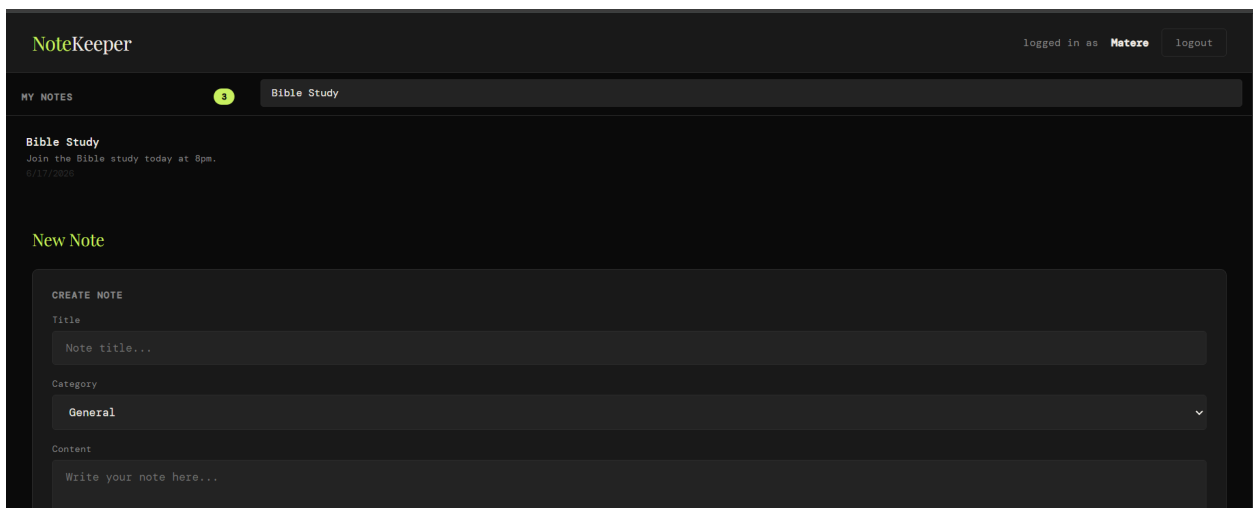


Fig 5: Edit Record

New Note

CREATE NOTE

Title

Tech Companies & Software

Category

Study

Content

Research about several tech companies in Nairobi.

Update Note

Fig 6: Delete Record

		id	user_id	title	content	category	created_at
☐	Edit	☐	065373d4-ef12-453c-9a9a-43778fa81694	Bible Study	Join the Bible study today at 8pm.	Personal	2026-06-17 08:29:58
☐	Edit	☐	4e52ad21-953c-494e-8829-fae1206837c6	Tech Companies & Software	Research about several tech companies in Nairobi.	Work	2026-06-17 08:29:36
☐	Edit	☐	82077cd8-5ae5-4865-83ec-692f802c00f3	Penetration Testing	Watch tutorials on how to Pen Test.	Study	2026-06-17 08:30:30

☐ Check all With selected: Edit Copy Delete Export

Fig 7: Responsive Design

Dimensions: iPhone 390 x 844 09 No Save

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Lighthouse

Recorder

What's new in DevTools 149

See all new features

Notes

MY NOTES

Bible Study

<!DOCTYPE html>

<html lang="en">

<head>

<body>

</body>

</html>

margin: 0 0 0 0

padding: 0 0 0 0

border: 0 0 0 0

box-sizing: border-box

color: #000000

display: flex

flex-basis: 0%

flex-grow: 1

flex-shrink: 1

font-family: "DM Mono", monospace

font-size: 14px

grid-template-columns: 1fr 1fr

grid-template-rows: 1fr 1fr

height: 783px

margin-bottom: 0px

margin-left: 0px

Fig 8: Form Validation

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

choose a username

Email

you@example.com

Password

.....

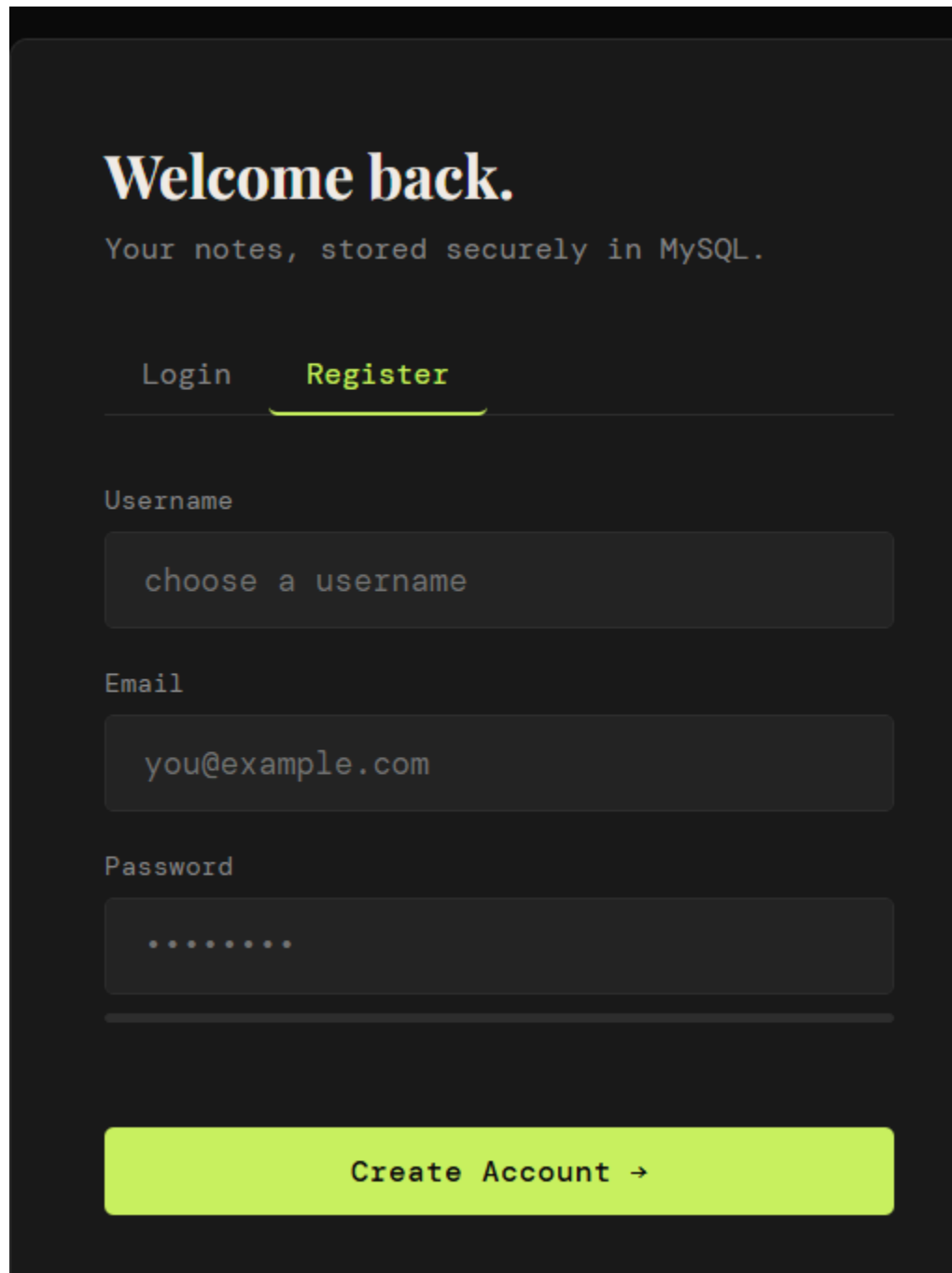
Create Account →

Please fill in all fields

Week 7: User Authentication and Session Management

Practical Task 1: Student Portal Login System

Fig 1: Registration

A registration form interface with a dark background. At the top, it says "Welcome back." in a large, bold, orange font, followed by "Your notes, stored securely in MySQL." in a smaller, light gray font. Below this, there are two tabs: "Login" and "Register". The "Register" tab is highlighted with a yellow underline. The form contains three input fields: "Username" with the placeholder text "choose a username", "Email" with the placeholder text "you@example.com", and "Password" with a series of dots. At the bottom, there is a large yellow button with the text "Create Account →".

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

choose a username

Email

you@example.com

Password

.....

Create Account →

Fig 2: Login

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

your username

Password

.....

Login →

Fig 3: Dashboard

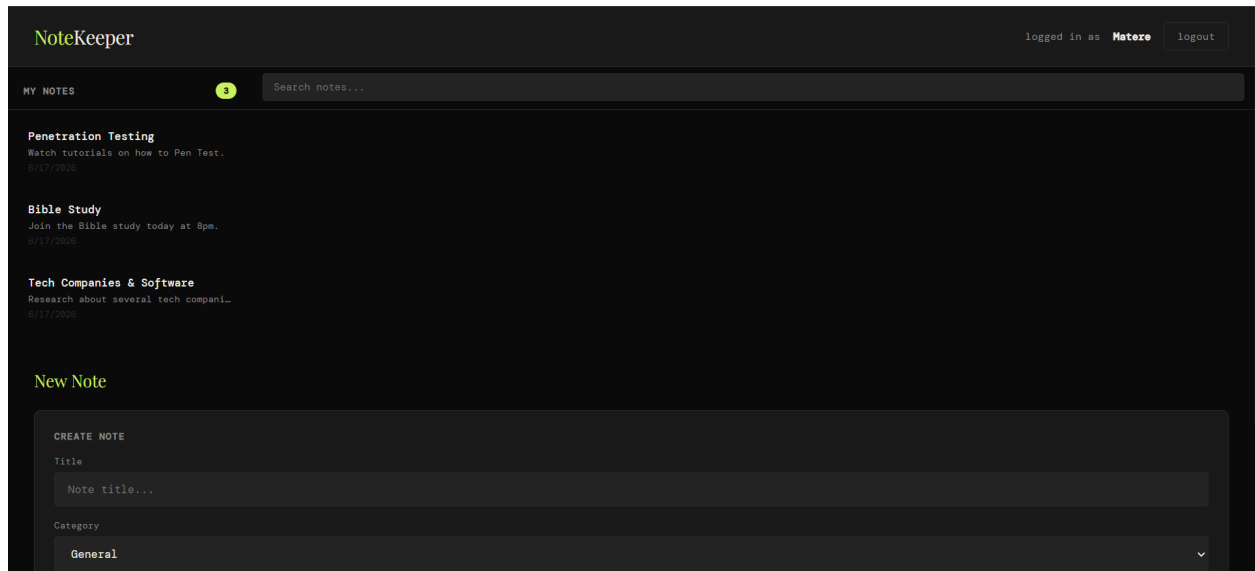


Fig 4: Logout

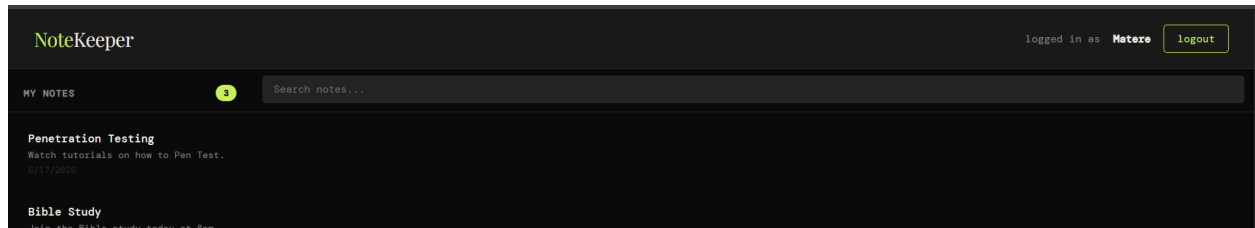
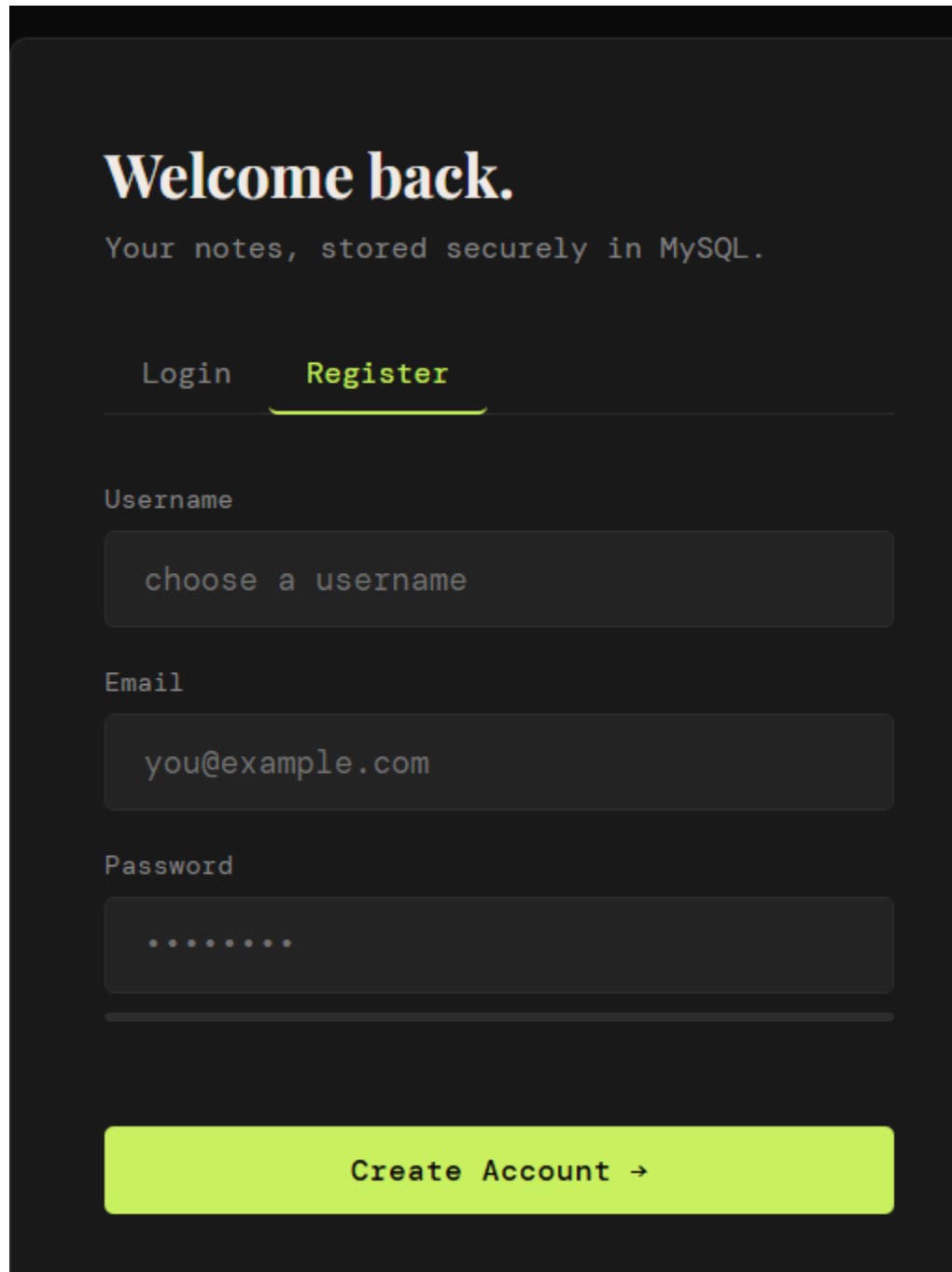


Fig 5: Database Script

> week1	71	<div class= sidebar
> week2	72	<div class="sideba
> week3	73	<h3>My Notes</h3
> week4	74	<span id="noteCo
> week5	75	</div>
JS server.js	76	<div class="notes-
.gitignore	77	<div class="empt
Notes Application Docum...	78	</div>
notes_app.sql U	79	</div>
package-lock.json	80	
	81	<div class="main-par
	82	<div class="panel-

Practical Task 2: Employee Portal

Fig 1: Registration



The image shows a registration form for an Employee Portal. The form is set against a dark background. At the top, it says "Welcome back." in a large, bold, white font, followed by "Your notes, stored securely in MySQL." in a smaller, white font. Below this, there are two tabs: "Login" and "Register". The "Register" tab is highlighted with a yellow underline. The form consists of three input fields: "Username" with the placeholder text "choose a username", "Email" with the placeholder text "you@example.com", and "Password" with a series of dots. Below the password field is a horizontal line. At the bottom of the form is a large yellow button with the text "Create Account →".

Welcome back.

Your notes, stored securely in MySQL.

Login Register

Username

choose a username

Email

you@example.com

Password

.....

Create Account →

Fig 2: Authentication

```
app.post('/api/login', async (req, res) => {
  const { username, password } = req.body;

  const [rows] = await db.execute(
    'SELECT id FROM users WHERE username = ? AND password = ?',
    [username, password]
  );
  if (rows.length === 0)
    return res.status(401).json({ error: 'Invalid username or password' });

  const token = uuidv4();
  await db.execute(
    'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
    [token, rows[0].id]
  );

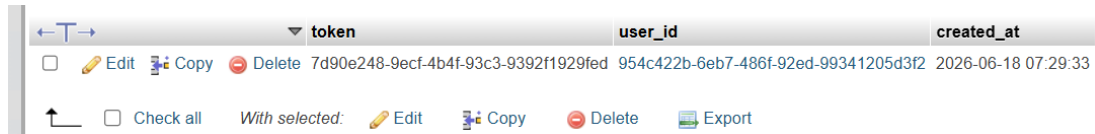
  res.json({ token, username });
});
```

Fig 3: Protected Page

```
124 app.post('/api/logout', requireAuth, async (req, res) => {
125   const token = req.headers['authorization'];
126   await db.execute('DELETE FROM sessions WHERE token = ?', [token]);
127   res.json({ message: 'Logged out' });
128 });
129
130 app.get('/api/notes', requireAuth, async (req, res) => {
131   const [rows] = await db.execute(
132     'SELECT * FROM notes WHERE user_id = ? ORDER BY created_at DESC',
133     [req.userId]
134   );
135   res.json(rows);
136 });
137
138 app.post('/api/notes', requireAuth, async (req, res) => {
139   const { title, content, category } = req.body;
140   if (!title || !content)
141     return res.status(400).json({ error: 'Title and content required' });
142
143   const id = uuidv4();
144   await db.execute(
145     'INSERT INTO notes (id, user_id, title, content, category) VALUES (?, ?, ?, ?, ?)',
146     [id, req.userId, title, content, category || 'General']
147   );
148   res.json({ id, title, content, category });
149 });
```

You, 22 hours ago • week6 - add category

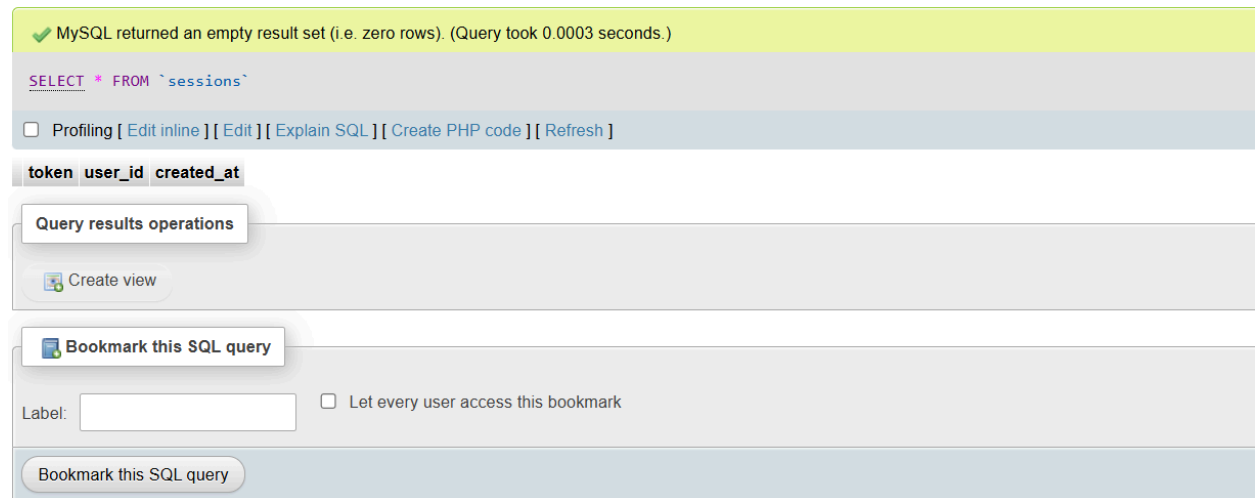
Fig 4: Session Management



	token	user_id	created_at
☐	7d90e248-9ecf-4b4f-93c3-9392f1929fed	954c422b-6eb7-486f-92ed-99341205d3f2	2026-06-18 07:29:33

↑ ☐ Check all With selected: Edit Copy Delete Export

Fig 5: Session Deleted on Logout



✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```
SELECT * FROM `sessions`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

token	user_id	created_at
-------	---------	------------

Query results operations

Create view

Bookmark this SQL query

Label: ☐ Let every user access this bookmark

Bookmark this SQL query

Practical Task 3: Multi-User Authentication System

Fig 1: Role Selection on Register

Welcome back.

Your notes, stored securely in MySQL.

Login

Register

Username

choose a username

Email

you@example.com

Role

Student



Student

Lecturer

Administrator

Create Account →

Fig 2: Register Three Users

				id	username	email	password	role	created_at
☐	Edit	Copy	Delete	1400bac7-527c-4d15-ba02-5d18eae54dac	JohnKiriaku	kiriakujohn@gmail.com	Player8-Ouch7-Recluse5-Sporty1-Flatbed6	administrator	2026-06-18 08:03:19
☐	Edit	Copy	Delete	1a6126e2-feb9-4c6b-a352-81effd5bad5c	BigYahu	yahuuben@gmail.com	Thorn1-Uranium6-Overhaul1-List7-Kung4	lecturer	2026-06-18 08:02:31
☐	Edit	Copy	Delete	679fa6a4-76f1-4ea4-b905-71d3d29613e6	Charles	charlesplays@gmail.com	Shelf7-Uptown5-Conductor8-Bubbly5-Handsfree5	student	2026-06-18 08:01:28

↑ ☐ Check all With selected: Edit Copy Delete Export

Fig 3: Role Badge on Dashboard

NoteKeeper logged in as BigYahu LECTURER logout

MY NOTES 0

No notes yet.
Create your first one →

New Note

CREATE NOTE

Title

Category

General

Content

Fig 4: Session Management

				token	user_id	created_at
☐	Edit	Copy	Delete	a04b8c86-706e-4ed0-a33e-136fce2b1a69	1a6126e2-feb9-4c6b-a352-81effd5bad5c	2026-06-18 08:04:21

↑ ☐ Check all With selected: Edit Copy Delete Export

Fig 5: Access Restriction

```
async function requireAuth(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).json({ error: 'No token provided' });

  const [rows] = await db.execute(
    'SELECT user_id FROM sessions WHERE token = ?', [token]
  );
  if (rows.length === 0) return res.status(401).json({ error: 'Invalid or expired session' });

  req.userId = rows[0].user_id;
  next();
}
```

Bonus Features

1. Remember Me: localStorage Keeps Session

Fig 1: Browser DevTools Application tab

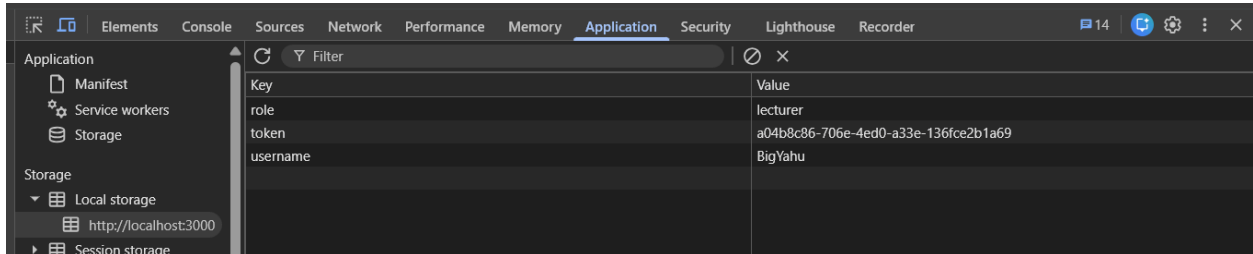
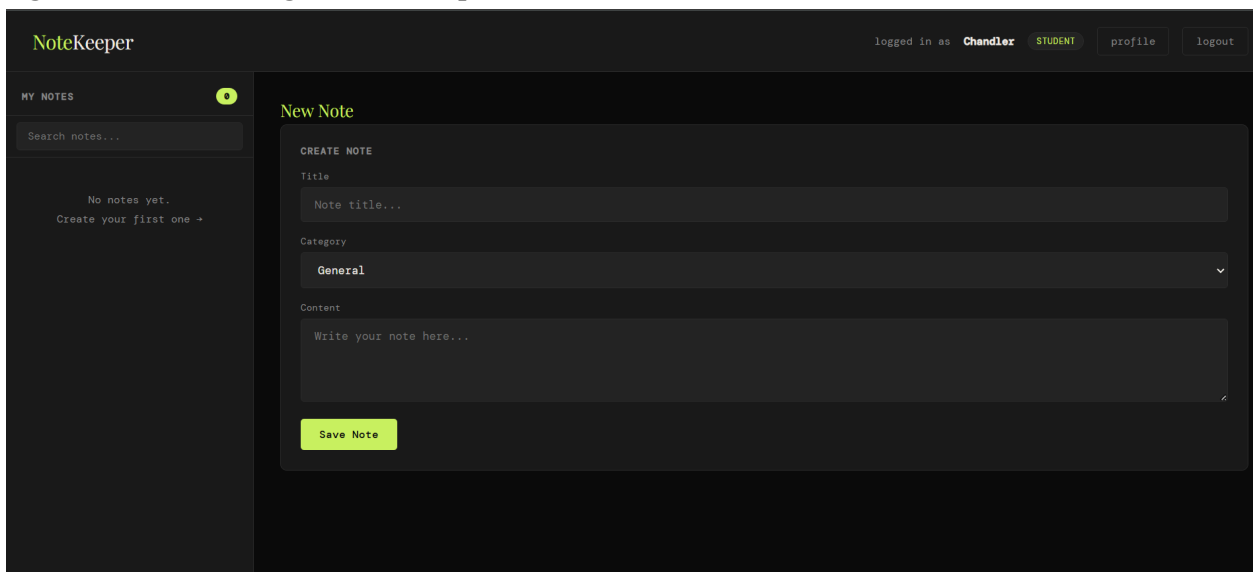


Fig 2: Browser showing the NoteKeeper dashboard



2. Email Verification

Fig 3: Browser showing the registration success message

Welcome back.

Your notes, stored securely in MySQL.

Login

Register

Forgot Password

Username

Chandler

Email

chandlermxtere@gmail.com

Role

Student



Password

.....

Strong

Create Account →

✓ Account created! Please check your email to verify your account.

Fig 4: Gmail inbox showing the NoteKeeper email verification message sent via Nodemailer using Gmail SMTP

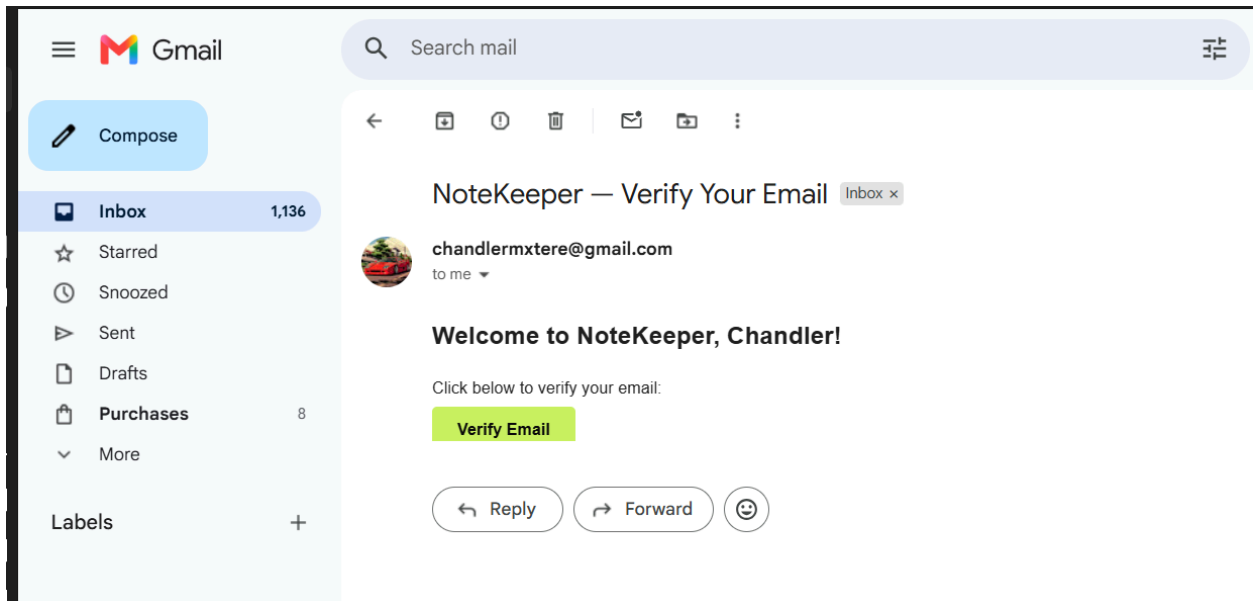


Fig 5: Browser showing the email verified confirmation page after clicking the verification link, confirming the token-based email verification system works correctly

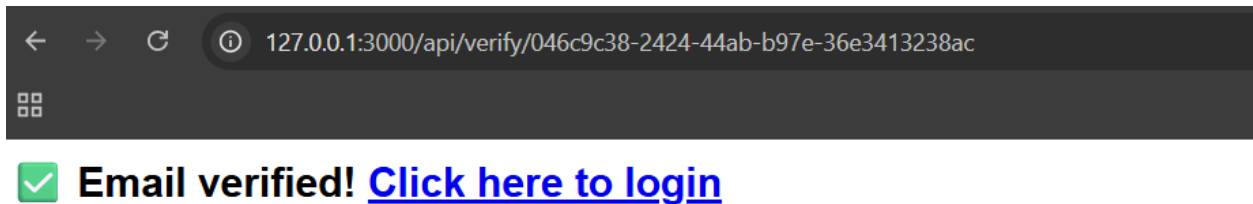
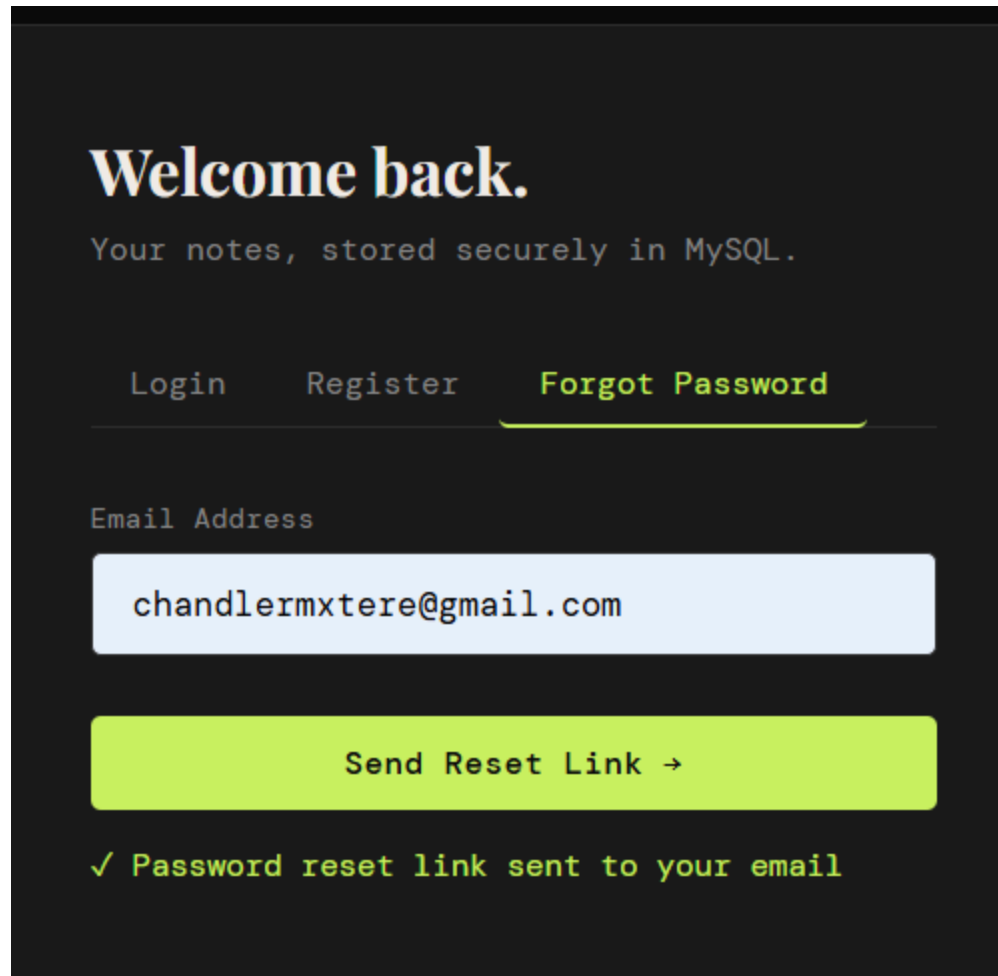


Fig 6: phpMyAdmin Browse view of the users table showing is_verified set to 1 confirming the verification status is updated and stored in the MySQL database

	id	username	email	password	role	is_verified	verify_token	reset_token	reset_expires	two_fa_secret	two_fa_enabled
☐	e584e197-60cf-466e-b593-850786fbfedb	Chandler	chandlermxtere@gmail.com	Pronto6-Art1-Retrieval4-Jockey7-Blizzard5	student	1	NULL	NULL	NULL	NULL	0

3. Password Reset

Fig 7: Browser showing the Forgot Password form with a success message confirming a password reset email was sent to the registered email address



The image shows a dark-themed web interface for a password reset. At the top, it says "Welcome back." in a large, bold, yellow font, followed by "Your notes, stored securely in MySQL." in a smaller, light gray font. Below this, there are three navigation links: "Login", "Register", and "Forgot Password". The "Forgot Password" link is highlighted with a yellow underline. Underneath the links, there is a label "Email Address" in a light gray font. Below the label is a light blue input field containing the email address "chandlermxtere@gmail.com". Below the input field is a yellow button with the text "Send Reset Link →". At the bottom, there is a yellow checkmark followed by the text "Password reset link sent to your email" in a yellow font.

Welcome back.

Your notes, stored securely in MySQL.

Login Register Forgot Password

Email Address

chandlermxtere@gmail.com

Send Reset Link →

✓ Password reset link sent to your email

Fig 8: Gmail inbox showing the NoteKeeper password reset email with the Reset Password link

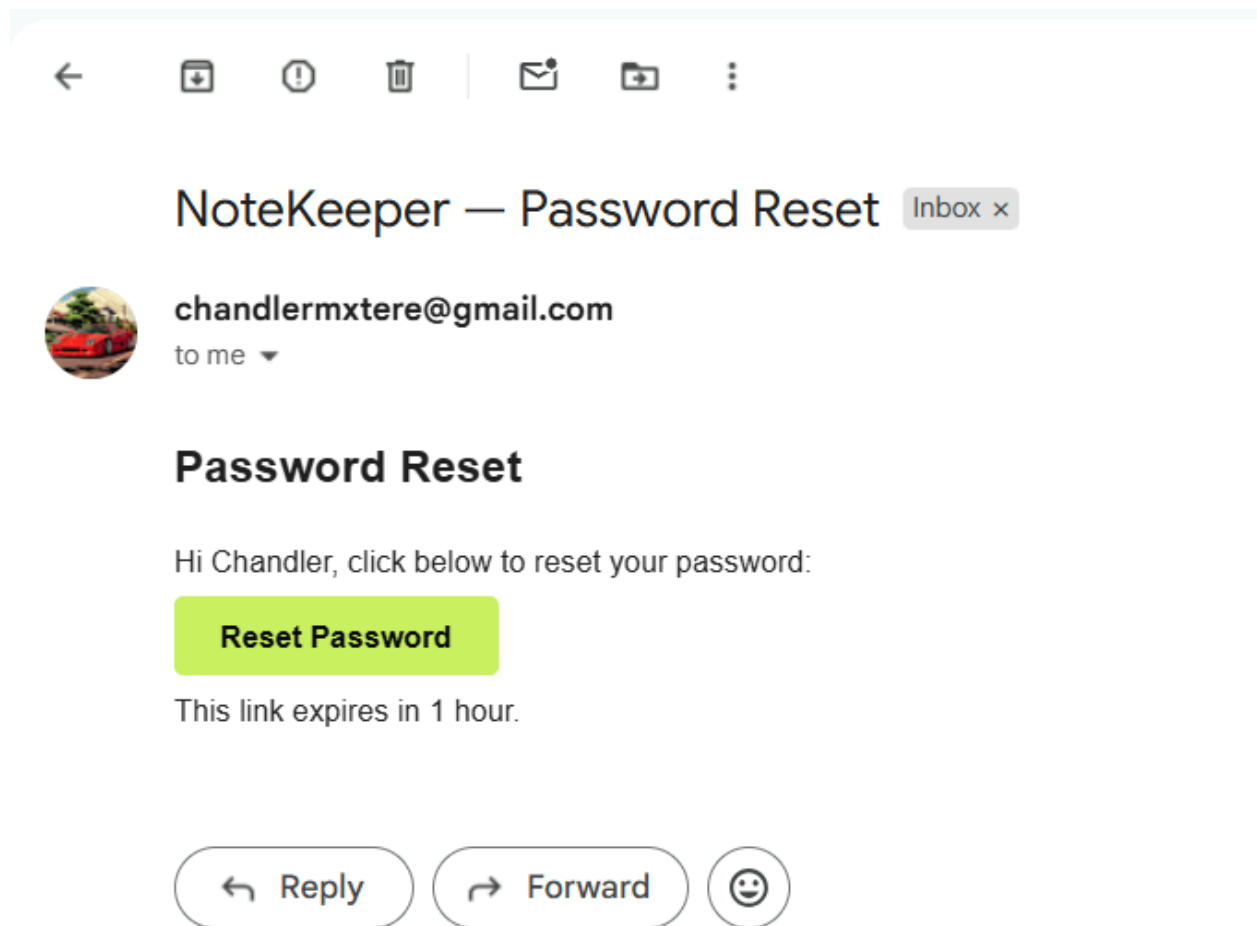
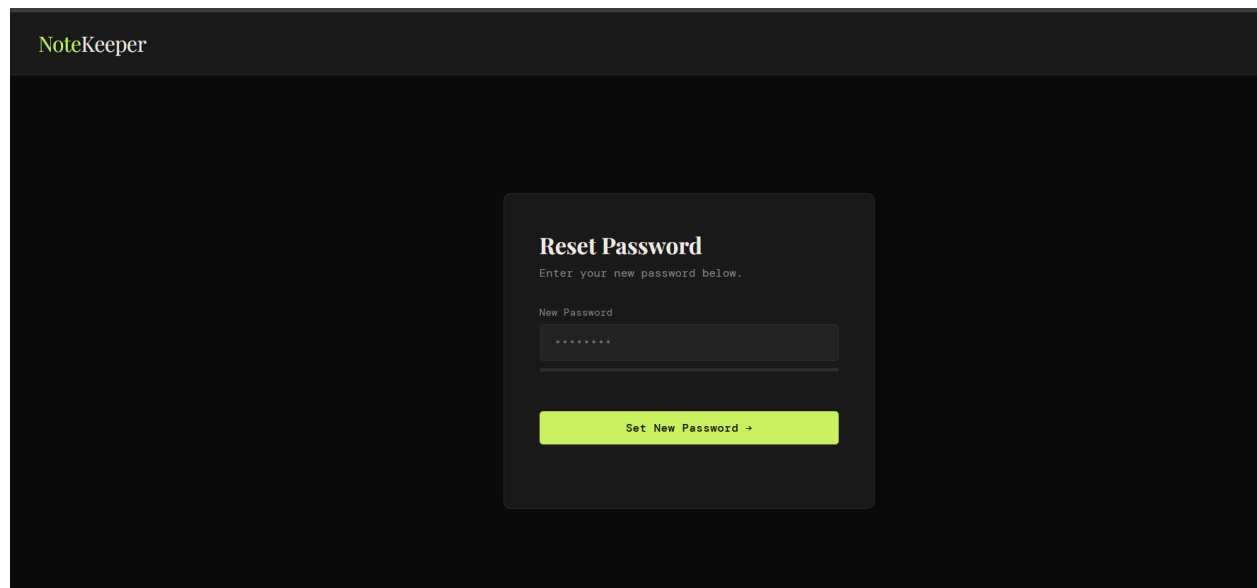


Fig 9: Browser showing the reset password page with the new password field and live strength indicator



4. 2FA: Google Authenticator

Fig 10: Browser showing the NoteKeeper profile panel with the QR code generated for Google Authenticator setup

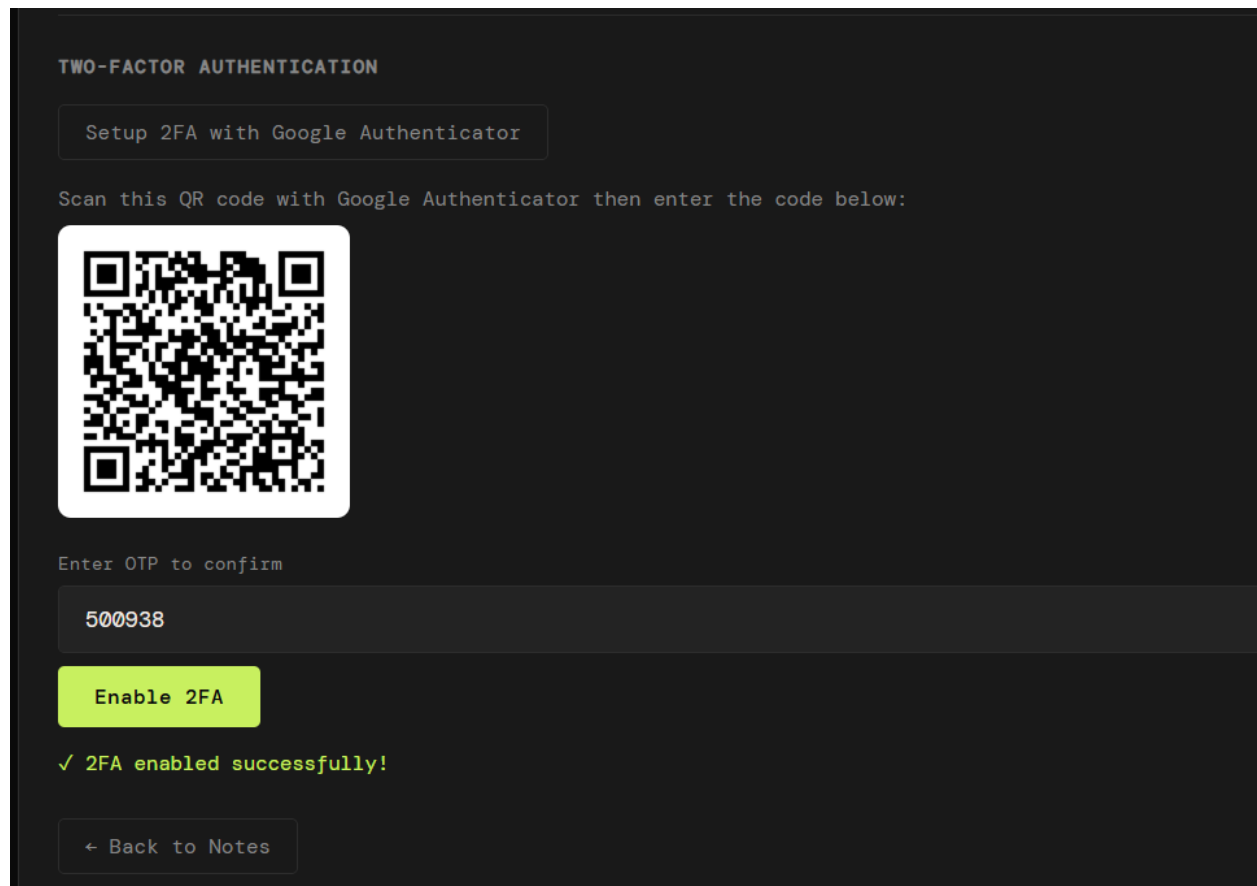


Fig 11: Browser showing the 2FA enabled success message after the OTP was verified and confirmed against the stored secret

TWO-FACTOR AUTHENTICATION

Setup 2FA with Google Authenticator

Scan this QR code with Google Authenticator then enter the code below:



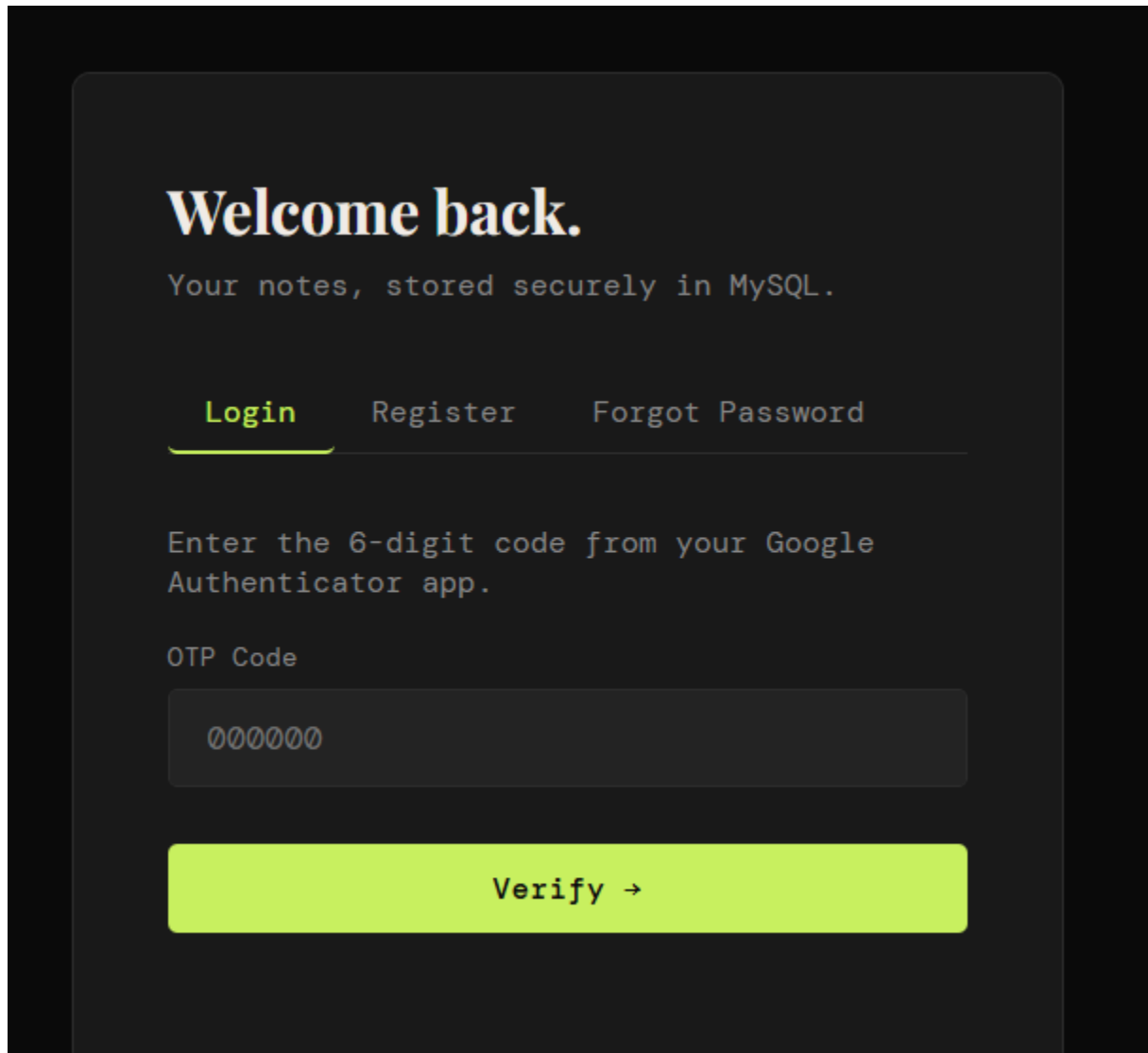
Enter OTP to confirm

787604

Enable 2FA

✓ 2FA enabled successfully!

Fig 12: Browser showing the OTP input screen appearing after username and password are verified, confirming 2FA is enforced before granting dashboard access



5. Profile Management

Fig 13: Browser showing the full profile panel displaying username, email, role, email verification status, 2FA status and account creation date

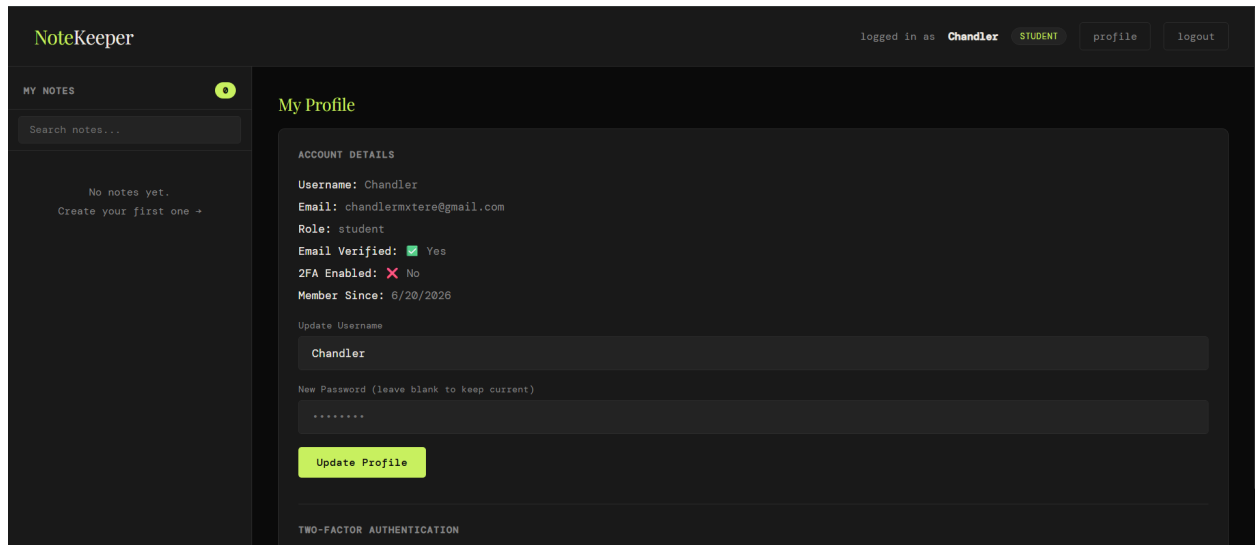


Fig 14: Browser showing the profile updated success message with the new username reflected in the header after submitting changes

